

URIB Settlement Flow Specification

Cross-Rail Settlement Lifecycle — Canonical Reference v1.0

Companion To: URIB Canonical Architecture Master Specification v1.0

Document Type: Normative Technical Specification

Status: NORMATIVE

Classification: Developer & Integrator Reference

Version: 1.0.0

Issued: 2026-07-10

RFC 2119 Compliance: Normative language (MUST, SHALL, SHOULD, MAY) used throughout per RFC 2119

Abstract.

This document is a precise, authoritative, and exhaustive specification of the complete settlement flow for the Universal Identity

&

Resource Bridge (URIB) system. It defines every state, every state transition, every actor role, every data structure, every timing constraint, every error path, and every recovery procedure involved in cross-rail settlement of canonical objects. The specification covers six settlement categories: simple single-rail settlement (S1), cross-rail two-party settlement (S2), multi-rail fan-out settlement (S3), optimistic settlement (S4), identity continuity settlement (S5), and disputed settlement (S6). All definitions, primitives, and architectural concepts introduced in the URIB Canonical Architecture Master Specification v1.0 are inherited by this document and extended specifically for the settlement domain. Implementors SHALL treat this document as the normative ground truth for all settlement behavior.

Table of Contents

1. Settlement Scope & Goals

1.1 What Settlement Means in URIB

1.2 Settlement Categories

1.3 Settlement Goals (Normative)

2. Settlement Actors & Roles

2.1 Actor Definitions

2.2 Responsibility Matrix

3. Settlement Data Structures

3.1 Settlement Request

3.2 Bridge Commitment (Settlement Context)

3.3 Settlement Proof

3.4 Finality Attestation Schema

3.5 Dispute Record Schema

3.6 Settlement Log Entry Schema

4. The Complete Settlement State Machine

4.1 State Definitions

4.2 Full State Transition Table

4.3 ASCII State Machine Diagram

4.4 Transition Guards (Formal)

5. Settlement Flows by Category

5.1 S1 — Intra-Rail Settlement Flow

5.2 S2 — Cross-Rail Bilateral Settlement Flow

5.3 S3 — Multi-Rail Fan-Out Settlement Flow

5.4 S4 — Optimistic Settlement Flow

5.5 S5 — Identity Continuity Settlement Flow

5.6 S6 — Disputed Settlement Flow

6. Rail-Specific Settlement Paths

6.1 Bitcoin 6.2 Ethereum 6.3 XRPL 6.4 Solana 6.5 IPFS 6.6 HTTPS/Off-Chain

7. Timing, Latency, and Timeout Model

8. Settlement Validation Procedures

9. Bridge Commitment Lifecycle (Settlement Context)

10. Settlement Proof Verification

11. Settlement Error Handling & Recovery

12. Settlement Observability & Auditing

13. Settlement API Reference

14. Settlement Security Considerations

15. Conformance & Testing

Appendices A – G

1. Settlement Scope & Goal

1.1 What Settlement Means in URIB

In the URIB protocol, **settlement** is defined as the irreversible, auditable confirmation that a canonical object's state transition has been acknowledged and finalized on one or more rails, and that a cryptographically verifiable proof of that finalization has been produced, persisted, and made publicly accessible. Settlement is the terminal phase of the Bridge Commitment lifecycle defined in the URIB Canonical Architecture Master Specification (hereafter: *the Master Spec*).

NORMATIVE DISTINCTION:

Settlement in URIB is NOT merely confirmation of an on-chain transaction. A transaction confirmed on a single chain satisfies only a necessary precondition. True URIB settlement MUST additionally satisfy ALL of the following: (1) canonical integrity of the canonical object is verified, (2) semantic consistency of the Semantic ID (SID) is confirmed across all target rails, (3) the ordinal is valid and non-duplicated, (4) all bridge commitments are finalized and anchored to the Taproot Root, and (5) the Taproot Root is published and provable on every target rail.

Settlement is the terminal state of a Bridge Commitment lifecycle. Prior to SETTLED, every canonical object in transit is considered *encumbered* and MUST NOT be operated on by any actor. After SETTLED, the canonical object is unencumbered and its new state is the authoritative ground truth across all participating rails.

The Settlement Engine is the URIB subsystem responsible for orchestrating the entire settlement lifecycle. It is stateful, durable, idempotent, and observable. All state transitions it performs MUST be logged immutably (see Section 12). The Settlement Engine exposes the API defined in Section 13 and reports metrics defined in Section 12.2.

1.2 Settlement Categories

URIB defines six settlement categories. Every settlement request MUST specify exactly one category. The category determines which flow is executed (see Section 5), which actors are involved, and which finality rules apply.

S1 — Intra-Rail Settlement

A canonical object's state transition is confirmed on a single rail with no cross-rail dependency. S1 is the simplest settlement form. It requires only one Rail Adapter, one Finality Watcher, and one bridge commitment leaf. S1 settlement is the baseline against which all other categories add complexity. The canonical object's source and target rail are identical; settlement confirms the state transition has been acknowledged by that rail's finality model. No Taproot Root cross-publication is required, but the Taproot Root MUST still be computed and anchored on the single rail for proof verifiability.

S2 — Cross-Rail Bilateral Settlement

A canonical object is bridged between exactly two distinct rails. Both rails MUST finalize before the settlement reaches SETTLED. S2 introduces the dual-finality requirement: the Settlement Engine MUST wait for Finality Watchers on both the source and destination rails to attest confirmation. The bridge commitment contains two leaves — one per rail — and the Taproot Root is published on both rails. S2 is the foundational cross-chain settlement form and is the most common settlement category in production URIB deployments.

S3 — Multi-Rail Fan-Out Settlement

A canonical object is settled simultaneously across $N > 2$ rails. All N rails MUST finalize before the settlement reaches SETTLED (unless overridden by `revert_on_partial_failure = false`). S3 uses a full Taproot tree with N leaves — one per rail. All Finality Watchers run concurrently; total settlement latency is bounded by the slowest rail. S3 is used for global identity propagation events, cross-ecosystem asset broadcasts, and canonical object state synchronization across heterogeneous rail ecosystems.

S4 — Optimistic Settlement

One or more rails' finality is accepted provisionally while other rails are still processing, or the entire settlement is optimistically accepted after the first rail confirms. S4 is subject to a challenge window during which any party may submit a counter-proof to dispute the settlement. If no valid challenge is received before the challenge window expires, the settlement transitions directly to SETTLED. If a valid challenge is received, the settlement transitions to CHALLENGED and then to ARBITRATION. S4 significantly reduces perceived settlement latency but introduces a time-bounded finality risk. S4 is only permitted when `settlement_options.optimistic = true` in the Settlement Request.

S5 — Identity Continuity Settlement

A ThreadZero event — specifically genesis, key rotation, fork, or tombstone — that MUST be settled across every rail on which the identity is currently active. S5 is the most operationally sensitive settlement category because it governs the authoritative identity state across all rails. S5 settlements require an `identity_continuity_proof` field in the Settlement Proof (see Section 3.3). All rails where the identity is registered MUST finalize before S5 reaches SETTLED. Partial finality is not permitted for S5 regardless of `revert_on_partial_failure`. S5 may only be initiated by the identity owner or a delegated key rotation authority.

S6 — Disputed Settlement

A settlement that has been challenged during an S4 challenge window or otherwise flagged by a validator. S6 requires arbitration and counter-proof submission. The Dispute Arbitrator evaluates the original settlement commitment and the challenger's counter-proof, then makes a deterministic resolution: either the settlement is valid (resume toward SETTLED) or the settlement is invalid (revert). S6 is the error-recovery category for adversarial conditions and protocol violations. See Section 5.6 for the full disputed settlement flow.

1.3 Settlement Goals (Normative)

The following goals are normative. All URIB settlement implementations **MUST** satisfy every goal for every settlement category.

1. **Atomicity.** Settlement **SHALL** either complete fully on all target rails or be fully reverted. Partial settlement — where some rails finalize and others do not — is only permitted when explicitly configured via `revert_on_partial_failure = false`, and even then, finalized rails **MUST** be marked distinctly and the overall settlement **MUST NOT** reach **SETTLED** state.
2. **Determinism.** Given identical inputs (Settlement Request, canonical object state, and rail states at submission time), the settlement engine **SHALL** always produce the same outputs and traverse the same state sequence. Non-determinism in settlement outcomes is a critical protocol violation.
3. **Auditability.** Every settlement action, including every state transition, every commitment write, every finality attestation, and every error, **SHALL** be logged with a cryptographically verifiable proof and a monotonic timestamp. Logs **MUST** be hash-chained (see Section 12.3).
4. **Liveness.** Settlement **MUST** eventually complete or explicitly fail. No settlement **SHALL** remain in a non-terminal state indefinitely. Every non-terminal state **MUST** have an associated timeout after which the engine transitions to a defined failure or recovery state (see Section 7.2).
5. **Non-repudiation.** Once a settlement reaches the **SETTLED** state and the Settlement Proof is published, no party — including the Initiator, any Rail Adapter, or the Settlement Engine itself — **SHALL** be able to deny that the settlement occurred. The Settlement Proof is the non-repudiable artifact.
6. **Rail Independence.** The settlement logic **SHALL NOT** depend on the continuous availability of any single rail. Rail Adapters **MUST** implement fault-tolerant connection patterns. A rail becoming temporarily unavailable **MUST** result in a timeout error, not a protocol hang or silent failure.

2. Settlement Actors & Roles

2.1 Actor Definitions

Initiator

Attribute	Definition
Role Name	Initiator
Description	The party that originates a settlement request. May be a human user, a smart contract, a protocol daemon, or an automated pipeline. The Initiator has custody over the canonical object prior to settlement and MUST sign the Settlement Request with its Ed25519 private key.
Permissions	Submit settlement requests; cancel DRAFT settlements; receive settlement proofs; submit challenges (as challenger).
Responsibilities	Construct a valid Settlement Request; provide valid initiator_sig; ensure expires_at is set appropriately; fund Settlement Escrow if required; monitor settlement status.
Trust Level	Untrusted by default. All Initiator claims are verified by Validators before proceeding. The Initiator's signature is necessary but not sufficient for settlement authorization.

Validator

Attribute	Definition
Role Name	Validator
Description	The party or set of parties responsible for verifying all pre-conditions before a settlement may proceed to the COMMITTING phase. Validators execute the full validation checklist defined in Section 8. In multi-validator deployments, a quorum (typically 2/3+1) MUST agree before validation passes.
Permissions	Read canonical object state; query ordinal registry; read rail capability declarations; reject settlements; flag potential conflicts.
Responsibilities	Execute all validation checks (Section 8); return pass/fail with error codes; sign validation attestation if passing.
Trust Level	Semi-trusted. Validators are permissioned participants whose public keys are registered in the URIB validator registry. Validator consensus (quorum) is required for high-value settlements.

Rail Adapter

Attribute	Definition
Role Name	Rail Adapter
Description	A protocol-specific module that translates URIB settlement operations into the native read/write primitives of a specific rail. One Rail Adapter instance exists per supported rail. Rail Adapters abstract all rail-specific details from the Settlement Engine. They implement a standard interface: <code>lock()</code> , <code>write_commitment()</code> , <code>publish_taproot()</code> , <code>query_finality()</code> , <code>revert()</code> .
Permissions	Read and write state on their assigned rail; lock/unlock canonical objects; publish Taproot Root; query block/ledger data.
Responsibilities	Implement rail-specific finality model; handle gas estimation (Ethereum), fee estimation (Bitcoin), ledger sequencing (XRPL); report errors with URIB error codes; maintain connection to ≥ 2 independent RPC endpoints.
Trust Level	Trusted infrastructure component. Rail Adapters run inside the URIB trust boundary and MUST be audited before deployment. Their keys are registered in the URIB component registry.

Ordinal Oracle

Attribute	Definition
Role Name	Ordinal Oracle
Description	The service that assigns and verifies ordinals across all rails. Ordinals provide globally monotonic sequencing of canonical object state transitions, as defined in the Master Spec. The Ordinal Oracle maintains the canonical ordinal registry and is the authoritative source for ordinal validity.
Permissions	Issue ordinals; invalidate stale ordinals; write to ordinal registry; read all canonical object histories.
Responsibilities	Enforce ordinal monotonicity; prevent ordinal reuse; sign ordinal proofs; maintain ordinal log.
Trust Level	Highly trusted. Ordinal Oracle compromise can break the monotonicity invariant. MUST be deployed with HSM-backed signing.

Taproot Aggregator

Attribute	Definition
Role Name	Taproot Aggregator
Description	The component that collects all bridge commitment leaves for a given <code>settlement_id</code> , sorts them deterministically, computes the Merkle Tree, and derives the Taproot Root (TR). The TR is then submitted to each Rail Adapter for publication on the target rails.
Permissions	Read bridge commitments; compute Merkle Trees; sign Taproot Root; submit TR to Rail Adapters.
Responsibilities	Deterministic leaf ordering (lexicographic by <code>commitment_id</code>); correct Merkle computation; TR signature; re-compute on reorg.
Trust Level	Trusted infrastructure. TR computation is deterministic and publicly verifiable; any party can re-derive the TR from the commitment leaves.

Settlement Escrow

Attribute	Definition
Role Name	Settlement Escrow
Description	The lockbox — implemented as a smart contract on EVM rails or as a URIB protocol primitive on non-EVM rails — that holds the canonical object's encumbered state during in-flight settlement. The Settlement Escrow issues bridge commitments on behalf of the Initiator, signs them with the Escrow key, and releases the object upon <code>SETTLED</code> or reverts upon <code>REVERTED</code> .
Permissions	Lock canonical objects; issue bridge commitments; release or revert canonical object state; slash bonds.
Responsibilities	Atomic lock-commit; bond custody; nullifier issuance on reversion.
Trust Level	Highly trusted. Settlement Escrow smart contract code MUST be formally verified and audited.

Finality Watcher

Attribute	Definition
Role Name	Finality Watcher
Description	The component that continuously polls each rail for finality confirmation of a published Taproot Root. One Finality Watcher instance operates per rail per in-flight settlement. Finality Watchers MUST use at least two independent RPC endpoints for Byzantine fault tolerance. When finality is confirmed, the Finality Watcher writes a signed Finality Attestation to the bridge commitment record.
Permissions	Query rail state (read-only); write Finality Attestations; trigger timeout events.
Responsibilities	Monitor confirmation depth; detect reorgs; write attestations; timeout escalation.
Trust Level	Trusted infrastructure. Finality Watcher attestations MUST be signed. Byzantine disagreement among Finality Watcher endpoints MUST escalate to the Settlement Engine.

Dispute Arbitrator

Attribute	Definition
Role Name	Dispute Arbitrator
Description	The component or DAO that resolves challenged settlements. The Dispute Arbitrator receives the original commitment, the challenger's counter-proof, and all relevant settlement evidence, then produces a deterministic resolution: VALID (settlement stands) or INVALID (settlement is reverted). On-chain arbitration is preferred for transparency and non-repudiability.
Permissions	Access all settlement records; request additional evidence; issue binding resolution; trigger bond slashing.
Responsibilities	Evaluate proofs; produce signed resolution within <code>arbitration_timeout</code> ; publish resolution on-chain where possible.
Trust Level	Trusted authority, but MUST be deterministic and auditable. Resolution logic MUST be open-source and version-pinned.

Auditor

Attribute	Definition
Role Name	Auditor
Description	A read-only observer that verifies completed settlements using the Settlement Proof and public rail data. Auditors have no write permissions and do not participate in the settlement flow. They independently verify Settlement Proofs using the algorithm in Section 10.1.
Permissions	Read settlement logs; read Settlement Proofs; query rail anchors; call POST /v1/settlements/verify.
Responsibilities	Independent verification; report anomalies; participate in compliance reporting.
Trust Level	Untrusted by the Settlement Engine (read-only access only). Auditors MUST NOT be granted write access under any circumstances.

2.2 Responsibility Matrix

Actor	Initiation	Validation	Commitment	Anchoring	Finality Watch	Settlement	Dispute
Initiator	Primary	Observer	N/A	N/A	Observer	Observer	Secondary
Validator	N/A	Primary	Observer	N/A	N/A	Observer	Secondary
Rail Adapter	N/A	Secondary	Primary	Primary	Secondary	Secondary	Secondary
Ordinal Oracle	N/A	Primary	Secondary	N/A	N/A	Secondary	Observer
Taproot Aggregator	N/A	N/A	Secondary	Primary	N/A	Secondary	Secondary
Settlement Escrow	N/A	N/A	Primary	N/A	N/A	Secondary	Primary
Finality Watcher	N/A	N/A	N/A	Observer	Primary	Primary	Observer
Dispute Arbitrator	N/A	N/A	N/A	N/A	N/A	Observer	Primary

Actor	Initiation	Validation	Commitment	Anchoring	Finality Watch	Settlement	Dispute
Auditor	Observer	Observer	Observer	Observer	Observer	Observer	Observer

Legend:

Primary = owns the phase and is accountable for its outcome. Secondary = participates in the phase but is not the accountable party. Observer = has read-only visibility but no action responsibility. N/A = not involved in this phase.

3. Settlement Data Structures

All data structures in this section are normative. Implementations **MUST** serialize and deserialize these structures exactly as specified. Field names are case-sensitive. All timestamps **MUST** be ISO 8601 UTC strings with millisecond precision (e.g., 2026-07-09T01:58:00.000Z). All hash fields **MUST** be lowercase hexadecimal strings.

3.1 Settlement Request

The Settlement Request is the entry-point data structure submitted by the Initiator to the Settlement Engine via POST /v1/settlements (see Section 13.1). It MUST be fully populated before submission; the engine will reject any request with missing required fields.

```
// SettlementRequest — Normative JSON Schema v1.0 { "settlement_id": "3f7a9c2e-1b4d-4e8f-
a0c5-d2e6f3a1b7c9", // TYPE: string (UUID v4) // REQUIRED: YES // VALID: Must be a valid
RFC 4122 UUID v4. Globally unique. // Used as idempotency key for all retry operations.
"settlement_category": "S2", // TYPE: string enum // REQUIRED: YES // VALID: One of: "S1",
"S2", "S3", "S4", "S5", "S6" // Determines which settlement flow is executed (Section 5).
"canonical_id": "c-hash:sha3-256:a3f1c9e2b7d4...", // TYPE: string (C-Hash) // REQUIRED:
YES // VALID: Must be a valid C-Hash as defined in Master Spec Section N. // Must reference
an existing canonical object. "semantic_id": "sid:urib:v1:token:fungible:erc20", // TYPE:
string (SID) // REQUIRED: YES // VALID: Must resolve to a registered semantic type in the SID
registry. // Semantic type must be compatible with all target rails. "ordinal": { "rail_id":
"ethereum:mainnet", // TYPE: string // REQUIRED: YES // VALID: Must be a registered
rail_id in the URIB rail registry. "epoch": 194, // TYPE: integer (uint64) // REQUIRED:
YES // VALID: Must be within the valid epoch window (see Section 8.6). "sequence": 10042 //
TYPE: integer (uint64) // REQUIRED: YES // VALID: Must be strictly greater than the last
known sequence // for this (rail_id, canonical_id) pair. }, "source_rail":
"bitcoin:mainnet", // TYPE: string (rail_id) // REQUIRED: YES // VALID: Must be a registered,
active rail id. "target_rails": ["ethereum:mainnet"], // TYPE: array of string (rail_id) //
REQUIRED: YES // VALID: Min 1 element. For S1, must equal [source_rail]. // For S2, must
have exactly 1 distinct rail (source != target). // For S3, must have >= 2 distinct rails. // All
rail_ids must be registered and active. "initiator_id": "did:urib:1:z6Mkf5r...Qvc", // TYPE:
string (DID or pubkey identifier) // REQUIRED: YES // VALID: Must be a resolvable DID or a
registered identity in URIB. "initiator_sig": "base64url:MEUCIQDh...", // TYPE: string
(Base64URL-encoded Ed25519 signature) // REQUIRED: YES // VALID: Ed25519 signature over:
// SHA3-256(settlement_id || canonical_id || ordinal.rail_id || ordinal.epoch ||
ordinal.sequence) // using the private key corresponding to initiator_id. "payload_hash":
"a3f1c9e2b7d4e6f8...", // TYPE: string (SHA-3-256 hex, 64 chars) // REQUIRED: YES // VALID:
SHA3-256 of the canonical object payload. // Must match recomputed hash at validation time.
"settlement_options": { "optimistic": false, // TYPE: boolean // REQUIRED: YES
// VALID: If true, category must be S4. If false, S4 is rejected. "challenge_window_seconds":
3600, // TYPE: integer // REQUIRED: YES if optimistic=true, OPTIONAL otherwise //
VALID: Min: 600 (10 min). Max: 86400 (24 hr). Default: 3600 (1 hr). "max_finality_wait_seconds":
7200, // TYPE: integer // REQUIRED: NO (optional) // VALID: Per-settlement override for
finality timeout. // Must not exceed expires_at - created_at. "revert_on_partial_failure": true
// TYPE: boolean // REQUIRED: NO (optional, default: true) // VALID: If true (default), partial
S3 failure reverts all rails. // If false, failed rails marked PENDING_RETRY. }, "created_at":
"2026-07-09T01:58:00.000Z", // TYPE: string (ISO 8601 UTC) // REQUIRED: YES // VALID:
Must be within ±30 seconds of engine wall-clock time. "expires_at": "2026-07-
09T04:58:00.000Z" // TYPE: string (ISO 8601 UTC) // REQUIRED: YES // VALID: Must be
strictly after created_at. // Must be far enough in the future to accommodate all rail //
finality windows if optimistic=false. }
```

3.2 Bridge Commitment (Settlement Context)

The Bridge Commitment is the core data structure that represents a single rail-pair commitment within a settlement. For S1, there is one commitment. For S2, there are two commitment leaves (one per ordered rail direction). For S3 with N rails, there are N commitment leaves. This schema extends the Bridge Commitment defined in the Master Spec with settlement-specific fields.

```
// BridgeCommitment (Settlement Context) — Normative JSON Schema v1.0 { "commitment_id":
"bc:urib:v1:9e3a7f2c1b5d...", // TYPE: string (derived: "bc:urib:v1:" + SHA3-256 of commitment
content) // REQUIRED: YES (system-generated; not provided by Initiator) "settlement_id":
"3f7a9c2e-1b4d-4e8f-a0c5-d2e6f3a1b7c9", // TYPE: string (UUID v4) // REQUIRED: YES //
VALID: Foreign key to the parent SettlementRequest. "canonical_id": "c-hash:sha3-
256:a3f1c9e2b7d4...", // TYPE: string (C-Hash) // REQUIRED: YES "rail_from":
"bitcoin:mainnet", // TYPE: string (rail_id) // REQUIRED: YES // VALID: Source rail for this
commitment leaf. "rail_to": "ethereum:mainnet", // TYPE: string (rail_id) // REQUIRED:
YES // VALID: Destination rail for this commitment leaf. // For S1: rail_from == rail_to.
"state_hash_from": "7c4e1a9f2b8d3e5c...", // TYPE: string (SHA3-256 hex) // REQUIRED:
YES // VALID: SHA3-256 of canonical object state on source rail at // commitment time.
Captured atomically with lock(). "state_hash_to": "f2a8c3e7d1b9f4a6...", // TYPE: string
(SHA3-256 hex) // REQUIRED: YES // VALID: SHA3-256 of expected canonical object state on
destination // rail after settlement. Verified on finality. "ordinal_from": { "rail_id":
"bitcoin:mainnet", "epoch": 193, "sequence": 20041 }, // TYPE: object (Ordinal) // REQUIRED:
YES "ordinal_to": { "rail_id": "ethereum:mainnet", "epoch": 194, "sequence": 10042 }, // TYPE:
object (Ordinal) // REQUIRED: YES "commitment_status": "PENDING", // TYPE: string enum
// REQUIRED: YES // VALID: One of: PENDING | ANCHORED | PROVEN | FINALIZED | //
SETTLED | REVERTED | DISPUTED // Lifecycle: PENDING → ANCHORED → PROVEN →
FINALIZED → SETTLED // Failure: any state → REVERTED // Dispute:
PENDING_FINALITY → DISPUTED "taproot_leaf_hash": "e5d3f1a7c2b9e4f6...", // TYPE:
string (SHA3-256 hex) // REQUIRED: YES (populated at ANCHORING phase) // VALID: SHA3-
256 of the serialized commitment leaf content. // This is the leaf value inserted into the Taproot
Merkle tree. "taproot_root": "9a2c7e5f3b1d8a4c...", // TYPE: string (SHA3-256 hex) //
REQUIRED: YES (populated at ANCHORING phase) // VALID: The Taproot Root computed by the
Taproot Aggregator. // Must match the on-chain published TR. "finality_attestation_from": {
"rail_id": "bitcoin:mainnet", "block_height": 850421, "block_hash":
"0000000000000000000000000000000000000000000000000000000000000000", "confirmation_depth": 6, "attested_at": "2026-07-
09T02:58:00.000Z", "watcher_sig": "base64url:MEQCIA..." }, // TYPE: object
(FinalityAttestation) // REQUIRED: YES (populated by Finality Watcher when finality confirmed) //
See Section 3.4 for full schema. "finality_attestation_to": { /* same structure */ }, // TYPE: object
(FinalityAttestation) // REQUIRED: YES (populated by Finality Watcher on destination rail)
"challenge_window_expires_at": "2026-07-09T03:58:00.000Z", // TYPE: string (ISO 8601 UTC) //
REQUIRED: YES if optimistic=true, NULL otherwise // VALID: Set at OPTIMISTIC state entry:
now() + challenge_window_seconds "created_at": "2026-07-09T01:58:05.123Z", // TYPE:
string (ISO 8601 UTC) // REQUIRED: YES (system-generated) "updated_at": "2026-07-
09T02:10:18.456Z" // TYPE: string (ISO 8601 UTC) // REQUIRED: YES (updated on every status
change) }
```


3.3 Settlement Proof

The Settlement Proof is the artifact produced when a settlement reaches the SETTLED state. It is the non-repudiable, independently verifiable record of the completed settlement. Every party MUST accept a valid Settlement Proof as definitive evidence of settlement. The proof MUST be published to the settlement log and made available via the API (Section 13.3).

```
// SettlementProof — Normative JSON Schema v1.0 {  "proof_id":      "sp:urib:v1:c7e9a3f1b2d5...",
// TYPE:    string (derived: "sp:urib:v1:" + proof_hash) // REQUIRED: YES (system-generated)
"settlement_id":    "3f7a9c2e-1b4d-4e8f-a0c5-d2e6f3a1b7c9", // TYPE:    string (UUID v4) //
REQUIRED: YES  "canonical_id":    "c-hash:sha3-256:a3f1c9e2b7d4...", // TYPE:    string (C-
Hash) // REQUIRED: YES  "settlement_category": "S2", // TYPE:    string enum
(S1|S2|S3|S4|S5|S6) // REQUIRED: YES  "taproot_root":    "9a2c7e5f3b1d8a4c...", // TYPE:
string (SHA-3-256 hex) // REQUIRED: YES // VALID:  Must match the TR published on all target
rails.  "commitment_leaves": [  "e5d3f1a7c2b9e4f6...",  "b7a4c2e9f1d3a8c5..." ], // TYPE:
array of string (SHA-3-256 hex) // REQUIRED: YES // VALID:  Ordered array of all
taproot_leaf_hash values for this settlement. //      Sorted lexicographically (same order used by
Taproot Aggregator). //      Merkle tree of these leaves must produce taproot_root.  "merkle_path": [
{ "sibling_hash": "d1a5e8c3f7b2a9e4...", "position": "right" }, { "sibling_hash":
"f4c2b7e9a1d3c6f8...", "position": "left" } ], // TYPE:    array of MerklePathNode // REQUIRED:
YES // VALID:  Merkle authentication path from commitment_leaves[0] to //      taproot_root.
Used for light client verification (Section 10.3).  "ordinal_proof": {  "ordinal": { "rail_id":
"ethereum:mainnet", "epoch": 194, "sequence": 10042 },  "prev_sequence": 10041,  "oracle_sig":
"base64url:MEUCIQDh...",  "issued_at": "2026-07-09T01:58:01.000Z" }, // TYPE:    object
(OrdinalProof) // REQUIRED: YES // VALID:  Signed ordinal issuance record from the Ordinal
Oracle. //      Proves non-duplication and monotonicity. See Appendix D.  "rail_anchors": {
"bitcoin:mainnet": {  "block_height": 850421,  "tx_hash":
"a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2",  "finality_status":
"FINALIZED",  "confirmation_depth": 6  },  "ethereum:mainnet": {  "block_height":
22847391,  "tx_hash":
"0xf9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d3c2b1a0f9e8",  "finality_status":
"FINALIZED",  "confirmation_depth": 2  } }, // TYPE:    object (map of rail_id → RailAnchor)
// REQUIRED: YES // VALID:  Must contain an entry for every rail in target_rails. //
finality_status must be "FINALIZED" for every entry.  "identity_continuity_proof": null, // TYPE:
object | null // REQUIRED: YES for S5 settlements; null for all others. // VALID:  Full ThreadZero
continuity chain from genesis to current event. //      See Section 5.5 for structure.  "witness_sigs": [
"base64url:MEQCIAX...",  "base64url:MEUCIQB..." ], // TYPE:    array of string (Base64URL
Ed25519 signatures) // REQUIRED: YES // VALID:  Signatures over proof_hash from: //      -
Settlement Escrow key (always required) //      - Taproot Aggregator key (always required) //      -
Ordinal Oracle key (always required) //      - Validator quorum keys (for multi-validator deployments)
"settled_at":    "2026-07-09T02:10:22.789Z", // TYPE:    string (ISO 8601 UTC) // REQUIRED:
YES // VALID:  Timestamp of SETTLED state entry.  "proof_hash":    "7f3a1c9e2b5d8f4a..." //
TYPE:    string (SHA-3-256 hex) // REQUIRED: YES // VALID:  SHA3-256 of the entire
SettlementProof document with //      this field set to the empty string "". //      Verification:
recompute and compare (see Section 10.1 Step 1). }
```

3.4 Finality Attestation Schema

```
// FinalityAttestation — Normative JSON Schema v1.0 { "attestation_id": "fa:urib:v1:3e7a9c2f...",
// TYPE: string (system-generated) "rail_id": "bitcoin:mainnet", // TYPE: string (rail_id)
"commitment_id": "bc:urib:v1:9e3a7f2c...", // TYPE: string (FK → BridgeCommitment)
"anchor_tx_hash": "a1b2c3d4e5f6...", // TYPE: string (hex) // VALID: The transaction hash of
the Taproot Root publication tx. "anchor_block_height": 850421, // TYPE: integer (uint64)
"anchor_block_hash": "00000000000000000002c1e...", // TYPE: string (hex)
"confirmation_depth": 6, // TYPE: integer // VALID: Number of blocks/epochs/ledgers
confirming the anchor tx. // Must meet or exceed the rail's URIB finality requirement.
"finality_model": "6-block-confirmation", // TYPE: string // VALID: The finality model
applied (see Section 6 per-rail definitions). "rpc_endpoints_used": [ "https://btc-rpc-
primary.urib.internal", "https://btc-rpc-secondary.urib.internal" ], // TYPE: array of string (URIs)
// VALID: Must have >= 2 entries (Byzantine fault tolerance requirement). "attested_at": "2026-
07-09T02:58:00.000Z", // TYPE: string (ISO 8601 UTC) "watcher_id":
"fw:urib:btc:mainnet:01", // TYPE: string (Finality Watcher component ID) "watcher_sig":
"base64url:MEQCIA..." // TYPE: string (Base64URL Ed25519 signature) // VALID: Ed25519
signature over: // SHA3-256(attestation_id || commitment_id || anchor_tx_hash ||
anchor_block_height || confirmation_depth || attested_at) }
```

3.5 Dispute Record Schema

```
// DisputeRecord — Normative JSON Schema v1.0 { "dispute_id": "dr:urib:v1:2c4e6a8f...",
"settlement_id": "3f7a9c2e-1b4d-4e8f-a0c5-d2e6f3a1b7c9", "commitment_id":
"bc:urib:v1:9e3a7f2c...", "challenger_id": "did:urib:1:z6Mkp9r...Abc", // TYPE: string (DID)
// VALID: Must be a registered identity. Challenger must post bond. "challenge_bond_tx":
"0xd3c4b5a6e7f8...", // TYPE: string (tx hash on bond rail) // VALID: Transaction locking the
challenge bond in the Settlement Escrow. // Minimum bond: 1% of settlement value or floor
amount. "counter_proof": { "proof_type": "state_hash_mismatch", "claimed_state_hash":
"f2a8c3e7d1b9f4a6...", "actual_state_hash": "c9e1a7f3b2d5e8c4...", "evidence_tx_hash":
"0xb8a7c6e5d4f3b2a1...", "evidence_rail": "ethereum:mainnet", "challenger_sig":
"base64url:MEUCIQDc..." }, // TYPE: object (CounterProof) // VALID: Must contain sufficient
evidence to invalidate the commitment. // proof_type one of: state_hash_mismatch |
ordinal_violation | // unauthorized_initiator | semantic_conflict "dispute_status":
"ARBITRATION", // TYPE: string enum // VALID: CHALLENGED | ARBITRATION |
RESOLVED_VALID | RESOLVED_INVALID "arbitration_started_at": "2026-07-
09T03:15:00.000Z", "arbitration_expires_at": "2026-07-10T03:15:00.000Z", "resolution": {
"outcome": "RESOLVED_INVALID", "reasoning": "state_hash_to in commitment does
not match...", "arbitrator_id": "arbitrator:urib:dao:v1", "resolution_tx_hash":
"0xf1e2d3c4b5a6...", "resolved_at": "2026-07-09T14:30:00.000Z", "arbitrator_sig":
"base64url:MEQCIBe..." }, // TYPE: object | null (null until dispute is resolved) "bond_outcome":
"SLASHED", // TYPE: string enum | null // VALID: null (pending) | RETURNED (challenger
won) | SLASHED (challenger wrong) "created_at": "2026-07-09T03:10:00.000Z",
"updated_at": "2026-07-09T14:30:05.000Z" }
```

3.6 Settlement Log Entry Schema

```
// SettlementLogEntry — Normative JSON Schema v1.0 { "log_id":
"log:urib:v1:00000000001847a3...", // TYPE: string (monotonic sequence ID, zero-padded 16 hex
digits) "prev_log_hash": "e3a1c9f7b2d4e6a8...", // TYPE: string (SHA-3-256 hex) // VALID:
SHA3-256 of the previous log entry. Chain-link for tamper evidence. // First entry: prev_log_hash
= SHA3-256("GENESIS"). "settlement_id": "3f7a9c2e-1b4d-4e8f-a0c5-d2e6f3a1b7c9",
"from_state": "COMMITTING", "to_state": "ANCHORING", "event":
"commitment_issued", "actor_id": "escrow:urib:mainnet:01", // TYPE: string (component or
user ID responsible for this transition) "actor_sig": "base64url:MEQCIB...", // TYPE: string
(Ed25519 signature by actor over log entry hash) "metadata": { "commitment_id":
"bc:urib:v1:9e3a7f2c...", "rail": "ethereum:mainnet", "error_code": null }, // TYPE:
object (arbitrary key-value metadata for this event) "timestamp": "2026-07-09T01:58:07.234Z",
// TYPE: string (ISO 8601 UTC, millisecond precision) "log_entry_hash": "f7a3c9e1b2d5f8a4..."
// TYPE: string (SHA-3-256 hex) // VALID: SHA3-256 of this log entry with log_entry_hash set to
"". // Verifiers MUST recompute and confirm this field on every entry. }
```

4. The Complete Settlement State Machine

4.1 State Definitions

The following states are normative. Every URIB Settlement Engine implementation MUST implement all states exactly as defined. States are represented in this document in ALLCAPS. Transitions between states are triggered by events and governed by guards (Section 4.4).

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
DRAFT	Settlement Request has been created by the Initiator but not yet submitted to the Settlement Engine.	Initiator constructs SettlementRequest locally.	Initiator calls POST /v1/settlements → Submit event.	No engine timeout. Initiator-side only. expires_at enforced at submission.
SUBMITTED	Settlement Request has been received by the Settlement Engine and	Valid HTTP 202 from engine; settlement_id logged.	Validation begins → VALIDATING. Duplicate detected → VALIDATION_FAILED (E102).	30s: if validation has not started, emit E600, → VALIDATION_FAILED.

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
	assigned a settlement_id. Engine has acknowledged receipt (HTTP 202) and queued validation.			
VALIDATING	Validators are executing all validation checks (Section 8): canonical integrity, semantic consistency, ordinal validity, signature verification, rail capability, and pre-condition checks.	Settlement successfully submitted; no duplicate detected.	All checks pass → COMMITTING. Any check fails → VALIDATION_FAILED.	30s (validation_timeout): → VALIDATION_FAILED with E106 if not resolved.
VALIDATION_FAILED	One or more validation checks failed. This is a terminal failure state. No retry is possible from this state. The Settlement Request MUST be corrected and	Any validation check returned a failure code.	Terminal. No exit. Archived after retention period.	N/A (terminal state).

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
	resubmitted with a new settlement_id.			
COMMITTING	The Settlement Escrow is issuing and recording bridge commitments. Canonical objects are being locked on the source rail(s). Bridge commitment records are being written to the commitment store.	VALIDATING completed with all checks passing.	All commitments issued → ANCHORING. Commitment write fails → COMMITMENT_FAILED.	60s per commitment: → COMMITMENT_FAILED (E200); retry up to 3x.
COMMITMENT_FAILED	Bridge commitment issuance failed. The Settlement Engine will attempt retry up to 3 times with exponential backoff. After 3 failed retries, the settlement transitions to REVERTING.	Commitment write failed or Escrow lock failed.	Retry success → COMMITTING (retry attempt). Max retries exceeded → REVERTING.	Retry timer governs backoff. See Section 7.3 and 11.2.

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
ANCHORING	The Taproot Aggregator is computing the Taproot Root from all commitment leaves and the Rail Adapters are publishing the TR to each target rail. This phase is complete when TR publication is confirmed on all rails.	All bridge commitments successfully issued.	TR published on all rails → PENDING_FINALITY. Any rail fails → ANCHORING_FAILED.	120s per rail (anchoring_timeout): → ANCHORING_FAILED (E300).
ANCHORING_FAILED	Taproot Root publication failed on one or more target rails. Retry is attempted. If all retries fail, the settlement transitions to REVERTING.	TR publication returned an error or timed out on any rail.	Retry success → ANCHORING. Max retries exceeded → REVERTING.	Retry backoff governs. After 3 failures → REVERTING.
PENDING_FINALITY	Taproot Root has been published on all target rails. Finality Watchers	TR anchored on all target rails.	All FWs attest finality → FINALIZING. Challenge received (S4) → CHALLENGED. Timeout → EXPIRED.	Rail-specific finality_timeout (see Section 7.2). Overall: expires_at.

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
	are monitoring each rail for finality confirmation. This state persists until all Finality Watchers have attested finality OR until finality_timeout is exceeded.			
OPTIMISTIC	(S4 only) At least one rail has finalized. The settlement is provisionally accepted as valid. The challenge window is open. Any party may submit a counter-proof to challenge the settlement.	optimistic=true AND at least one rail has confirmed finality.	Challenge received → CHALLENGED. Window expires with no challenge → FINALIZING (→ SETTLED).	challenge_window_seconds: window expiry → FINALIZING.
CHALLENGED	A dispute has been raised by a challenger who has submitted a counter-proof and posted a	Valid challenge submission received during OPTIMISTIC window.	Always → ARBITRATION immediately on challenge validation.	30s to validate challenge submission; if invalid → back to OPTIMISTIC.

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
	challenge bond. The settlement is paused pending arbitration.			
ARBITRATION	The Dispute Arbitrator is evaluating the original commitment and the challenger's counter-proof. All settlement activity is paused. The Arbitrator has until arbitration_timeout to produce a resolution.	Valid challenge received; Dispute Record created.	Resolution VALID → ARBITRATION_RESOLVED_VALID. Resolution INVALID → ARBITRATION_RESOLVED_INVALID.	86400s (24hr): → escalate to admin. Settlement remains in ARBITRATION until resolved or admin action.
ARBITRATION_RESOLVED_VALID	The Dispute Arbitrator has confirmed the settlement is valid. The challenger's bond is slashed. Settlement resumes from PENDING_FINALITY.	Arbitrator resolution = VALID.	→ PENDING_FINALITY (resume watching remaining finalities).	Auto-transition after bond slashing completes (< 30s).

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
ARBITRATION_RESOLVED_INVALID	The Dispute Arbitrator has rejected the settlement. The Initiator is penalized. Settlement transitions to REVERTING.	Arbitrator resolution = INVALID.	→ REVERTING.	Auto-transition immediately after resolution.
FINALIZING	All target rails have confirmed finality. The Settlement Engine is assembling the Settlement Proof from all finality attestations, ordinal proofs, commitment leaves, and merkle paths.	All Finality Watchers have attested finality for all target rails.	Proof assembly succeeds → SETTLED. Proof assembly fails → retry (E602); max retries → REVERTING.	60s for proof assembly; if exceeded, retry or alert.
SETTLED	Terminal success state. Settlement Proof has been assembled, signed, persisted, and published. All bridge commitments are marked	Settlement Proof assembled and all witness signatures collected.	Terminal. → ARCHIVED after retention period (configurable, default 7 years).	N/A (terminal state).

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
	SETTLED. The canonical object is unencumbered on all target rails with its new state authoritative.			
REVERTING	The settlement is being rolled back. Rail Adapters are reverting state on all rails that had committed state. Nullifiers are being issued to the ordinal registry. The source canonical object is being unlocked and returned to its pre-settlement state.	COMMITMENT_FAILED (retries exhausted), ANCHORING_FAILED (retries exhausted), ARBITRATION_RESOLVED_INVALID, or EXPIRED (if already committed).	Reversion complete → REVERTED. Reversion itself fails → alert; retry reversion.	Reversion MUST complete. No timeout on REVERTING; escalate if exceeds 5 minutes.
REVERTED	Terminal failure/rollback state. All committed state on all rails has been restored to	REVERTING completed successfully.	Terminal. Retained in settlement log permanently.	N/A (terminal state).

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
	pre-settlement values. Nullifier is recorded. Initiator is notified with the failure code.			
EXPIRED	The settlement timed out (expires_at exceeded) before reaching SETTLED or REVERTED. If commitments had been issued, EXPIRED triggers REVERTING. If no commitments were issued, EXPIRED is the terminal state.	expires_at timestamp exceeded while in any non-terminal state.	If committed: → REVERTING. If not committed: terminal EXPIRED.	N/A (timeout is the trigger for this state).
ARCHIVED	A successfully SETTLED settlement that has been moved to long-term cold storage after the active	SETTLED state + retention period elapsed (default: 90 days for hot storage).	Terminal for active storage. Proof retrieval still available.	N/A.

State	Meaning	Entry Conditions	Exit Conditions	Timeout Behavior
	retention period. Settlement Proofs remain retrievable. Not shown in active settlement dashboards.			

4.2 Full State Transition Table

From State	Event	To State	Actor Responsible	Condition	Side Effect
DRAFT	submit()	SUBMITTED	Initiator	Request schema valid; not expired; settlement_id unique	Log entry written; HTTP 202 returned
SUBMITTED	validation_start()	VALIDATING	Settlement Engine	No duplicate settlement_id for canonical_id	Validators invoked; ordinal reserved
SUBMITTED	duplicate_detected()	VALIDATION_FAILED	Settlement Engine	Active settlement exists for canonical_id	Error E102 returned; log entry written
VALIDATING	validate_ok()	COMMITTING	Validator	All 7 validation checks pass (Section 8)	Validation attestation signed; commitment issuance begins
VALIDATING	validate_fail()	VALIDATION_FAILED	Validator	Any validation	Error code logged;

From State	Event	To State	Actor Responsible	Condition	Side Effect
				check returns error code	Initiator notified; ordinal reservation released
VALIDATING	timeout()	VALIDATION_FAILED	Settlement Engine	validation_timeout (30s) exceeded	E106 logged; ordinal released
COMMITTING	commitment_issued()	ANCHORING	Settlement Escrow	All bridge commitment records written atomically; canonical object locked	commitment_status → PENDING; Taproot Aggregator invoked
COMMITTING	commitment_write_failed()	COMMITMENT_FAILED	Settlement Escrow	DB write failed or Escrow lock failed	E200 logged; retry counter incremented
COMMITMENT_FAILED	retry_commitment()	COMMITTING	Settlement Engine	retry_count < 3; backoff wait elapsed	Retry attempt logged with attempt number
COMMITMENT_FAILED	max_retries_exceeded()	REVERTING	Settlement Engine	retry_count ≥ 3	Reversion initiated; E200 final logged
ANCHORING	taproot_published()	PENDING_FINALITY	Taproot Aggregator + Rail Adapters	TR published on all target rails; commitment_status → ANCHORED	Finality Watchers activated for all rails

From State	Event	To State	Actor Responsible	Condition	Side Effect
ANCHORING	anchor_failed()	ANCHORING_FAILED	Rail Adapter	TR publication failed or timed out on any rail	E301 logged with rail_id; retry counter incremented
ANCHORING_FAILED	retry_anchor()	ANCHORING	Settlement Engine	retry_count < 3; backoff wait elapsed	Re-anchor attempt logged
ANCHORING_FAILED	max_retries_exceeded()	REVERTING	Settlement Engine	retry_count ≥ 3	Reversion initiated
PENDING_FINALITY	all_finalities_confirmed()	FINALIZING	Finality Watcher(s)	All rails have confirmed finality; all attestations signed	commitment_status → FINALIZED; proof assembly begins
PENDING_FINALITY	optimistic_first_rail()	OPTIMISTIC	Finality Watcher	optimistic=true AND first rail finalized	Challenge window opened; timer started
PENDING_FINALITY	challenge_received()	CHALLENGED	Challenger (Initiator role)	Valid challenge with bond during any open window	Dispute Record created; challenge bond locked
PENDING_FINALITY	finality_timeout()	EXPIRED	Settlement Engine	Rail-specific finality_timeout exceeded OR expires_at exceeded	E400 logged; if committed → REVERTING

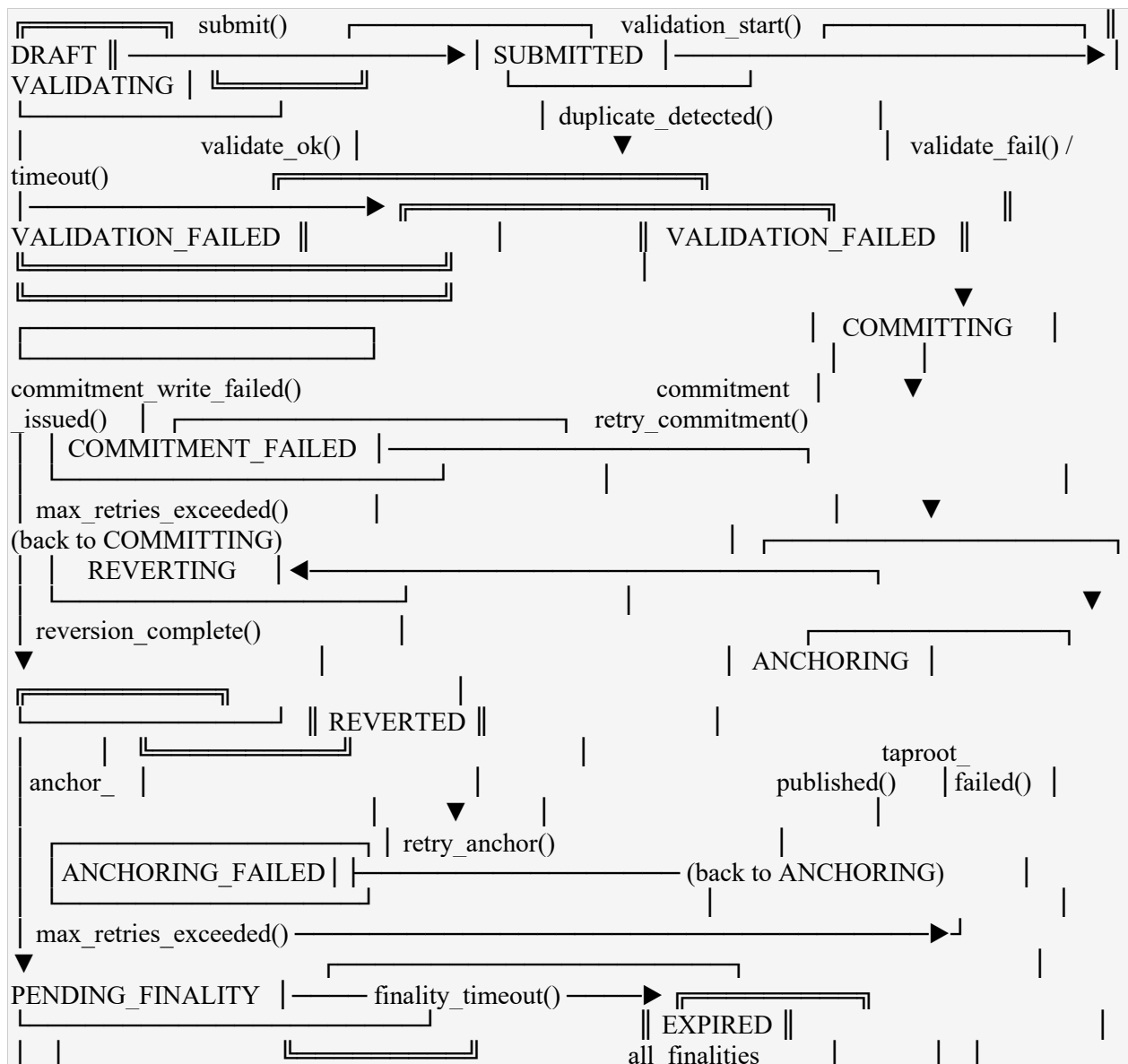
From State	Event	To State	Actor Responsible	Condition	Side Effect
OPTIMISTIC	challenge_window_expired()	FINALIZING	Settlement Engine	challenge_window_seconds elapsed with no valid challenge	commitment_status → FINALIZED; proof assembly begins
OPTIMISTIC	challenge_received()	CHALLENGED	Challenger	Valid challenge with sufficient bond received before window expires	Dispute Record created; challenge bond locked; window paused
CHALLENGED	challenge_validated()	ARBITRATION	Settlement Engine	Challenge submission is structurally valid; bond confirmed	Arbitrator invoked; arbitration_expires_at set
CHALLENGED	challenge_invalid()	OPTIMISTIC	Settlement Engine	Challenge submission invalid (bad format, insufficient bond)	Challenge rejected; bond returned; window resumes
ARBITRATION	resolution_valid()	ARBITRATION_RESOLVED_VALID	Dispute Arbitrator	Arbitrator determines original commitment is correct	Challenger bond slashed; Dispute Record updated
ARBITRATION	resolution_invalid()	ARBITRATION_RESOLVED_INVALID	Dispute Arbitrator	Arbitrator determines commitment is invalid	Initiator penalized; Dispute Record updated
ARBITRATION_RESOLVED_VALID	resume()	PENDING_FINALITY	Settlement Engine	Bond slashing complete	Finality watching resumes; log entry written

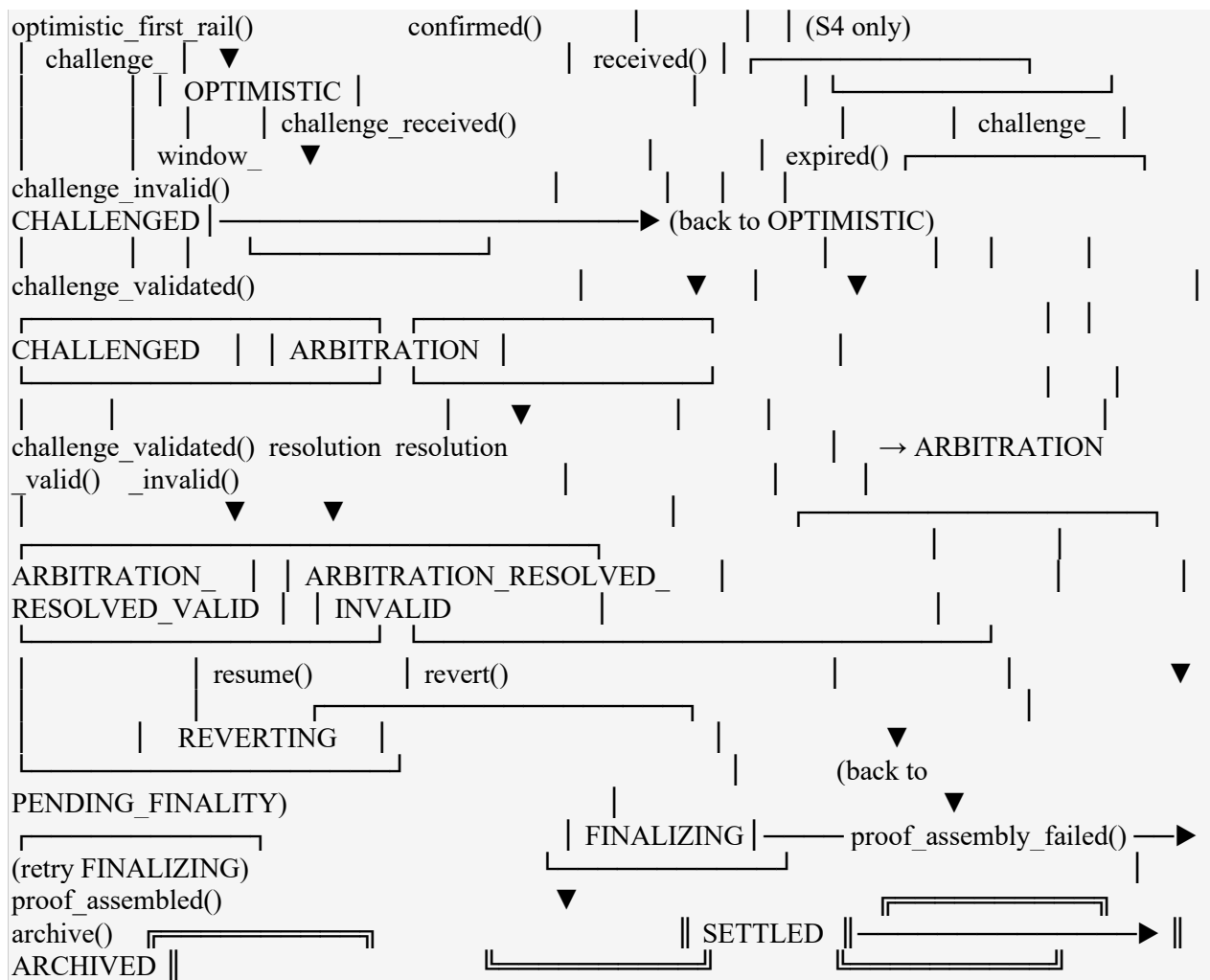
From State	Event	To State	Actor Responsible	Condition	Side Effect
ARBITRATION_RESOLVED_INVALID	revert()	REVERTING	Settlement Engine	Arbitration resolution signed and published	Reversion initiated
FINALIZING	proof_assembled()	SETTLED	Settlement Engine	Settlement Proof valid; all witness_sigs collected	commitment_status → SETTLED; proof published; Initiator notified
FINALIZING	proof_assembly_failed()	FINALIZING	Settlement Engine	E602; retry proof assembly	Retry logged; alert if repeated
REVERTING	reversion_complete()	REVERTED	Rail Adapters + Settlement Escrow	All rails reverted; nullifier issued; canonical object unlocked	commitment_status → REVERTED; Initiator notified with error
SETTLED	archive()	ARCHIVED	Settlement Engine (automated)	Active retention period elapsed	Data moved to cold storage; index entry retained
any non-terminal	expires_at_exceeded()	EXPIRED	Settlement Engine	Wall clock > expires_at	E106; if committed → REVERTING first

4.3 ASCII State Machine Diagram

Diagram Note:

The following ASCII state machine diagram represents the complete URIB settlement state machine. States are enclosed in rectangular boxes. Transitions are labeled with triggering events. The happy path is the topmost horizontal flow. Failure and recovery paths branch downward. The optimistic and dispute paths are shown in the lower section. All terminal states are double-bordered.





4.4 Transition Guards (Formal)

The following transition guards are normative boolean predicates that MUST evaluate to TRUE before a state transition is permitted. Implementations MUST enforce these guards atomically. A guard evaluation failure MUST produce the appropriate error code and MUST NOT cause a partial transition.

Transition	Guard Predicate (Formal)	Failure Action
DRAFT → SUBMITTED	<code>is_valid_uuid_v4(settlement_id) == TRUE AND schema_valid(request) == TRUE AND now() < expires_at AND settlement_id NOT IN global_settlement_registry</code>	HTTP 400 or 409 with error code
SUBMITTED → VALIDATING	<code>no_active_settlement_exists(canonical_id) == TRUE AND settlement_id_unique(settlement_id) == TRUE AND now() < expires_at</code>	→ VALIDATION_FAILED (E102)
VALIDATING → COMMITTING	<code>canonical_integrity_check(canonical_id, payload_hash) == TRUE AND semantic_consistency_check(semantic_id, target_rails) == TRUE AND ordinal_valid(ordinal, canonical_id) == TRUE AND initiator_sig_valid(initiator_sig, settlement_id, canonical_id, ordinal) == TRUE AND rails_all_active(target_rails) == TRUE AND no_conflicting_commitment(canonical_id) == TRUE AND ordinal_fresh(ordinal) == TRUE AND expires_at_compatible_with_finality(target_rails, expires_at) == TRUE</code>	→ VALIDATION_FAILED with specific error code (E100–E107)
COMMITTING → ANCHORING	<code>all_commitments_written(settlement_id) == TRUE AND canonical_object_locked(canonical_id, source_rail) == TRUE AND count(commitment_records) == expected_commitment_count(settlement_category, target_rails) == TRUE</code>	→ COMMITMENT_FAILED (E200)
ANCHORING → PENDING_FINALITY	<code>taproot_root_computed(settlement_id) == TRUE AND FOR ALL rail IN target_rails: taproot_published_on_rail(rail, taproot_root) == TRUE AND</code>	→ ANCHORING_FAILED (E301)

Transition	Guard Predicate (Formal)	Failure Action
	<code>commitment_status_all(settlement_id, "ANCHORED") == TRUE</code>	
PENDING_FINALITY → FINALIZING	FOR ALL rail IN target_rails: <code>finality_attested(rail, settlement_id) == TRUE AND</code> <code>finality_attestation_sig_valid(rail, settlement_id) == TRUE AND</code> <code>commitment_status_all(settlement_id, "FINALIZED") == TRUE</code>	Remain in PENDING_FINALITY; emit alert if timeout approaching
PENDING_FINALITY → OPTIMISTIC	<code>settlement_options.optimistic == TRUE</code> AND EXISTS rail IN target_rails: <code>finality_attested(rail, settlement_id) == TRUE AND now() < expires_at</code>	Remain in PENDING_FINALITY
OPTIMISTIC → CHALLENGED	<code>now() < challenge_window_expires_at</code> AND <code>challenge_bond_confirmed(challenge_bond_tx) == TRUE AND</code> <code>counter_proof_structurally_valid(counter_proof) == TRUE AND</code> <code>challenger_id_registered(challenger_id) == TRUE</code>	Challenge rejected; bond returned; E500 or E501
ARBITRATION → ARBITRATION_RESOLVED_VALID	<code>arbitrator_sig_valid(resolution, arbitrator_id) == TRUE AND</code> <code>resolution.outcome == "VALID" AND</code> <code>now() < arbitration_expires_at</code>	Remain in ARBITRATION; escalate if arbitration_timeout exceeded
FINALIZING → SETTLED	<code>proof_hash_valid(settlement_proof) == TRUE AND count(witness_sigs) ≥</code> <code>required_witness_count(deployment_mode) AND FOR ALL rail IN target_rails:</code> <code>rail_anchor_verified(rail, settlement_id) == TRUE AND</code> <code>commitment_status_all(settlement_id, "FINALIZED") == TRUE</code>	→ retry FINALIZING; alert E602
REVERTING → REVERTED	FOR ALL rail IN committed_rails: <code>rail_state_reverted(rail, canonical_id) == TRUE AND nullifier_recorded(ordinal, canonical_id) == TRUE AND</code> <code>canonical_object_unlocked(canonical_id, source_rail) == TRUE</code>	Remain in REVERTING; escalate if exceeds 5 minutes

5. Settlement Flows by Category

5.1 S1 — Intra-Rail Settlement Flow

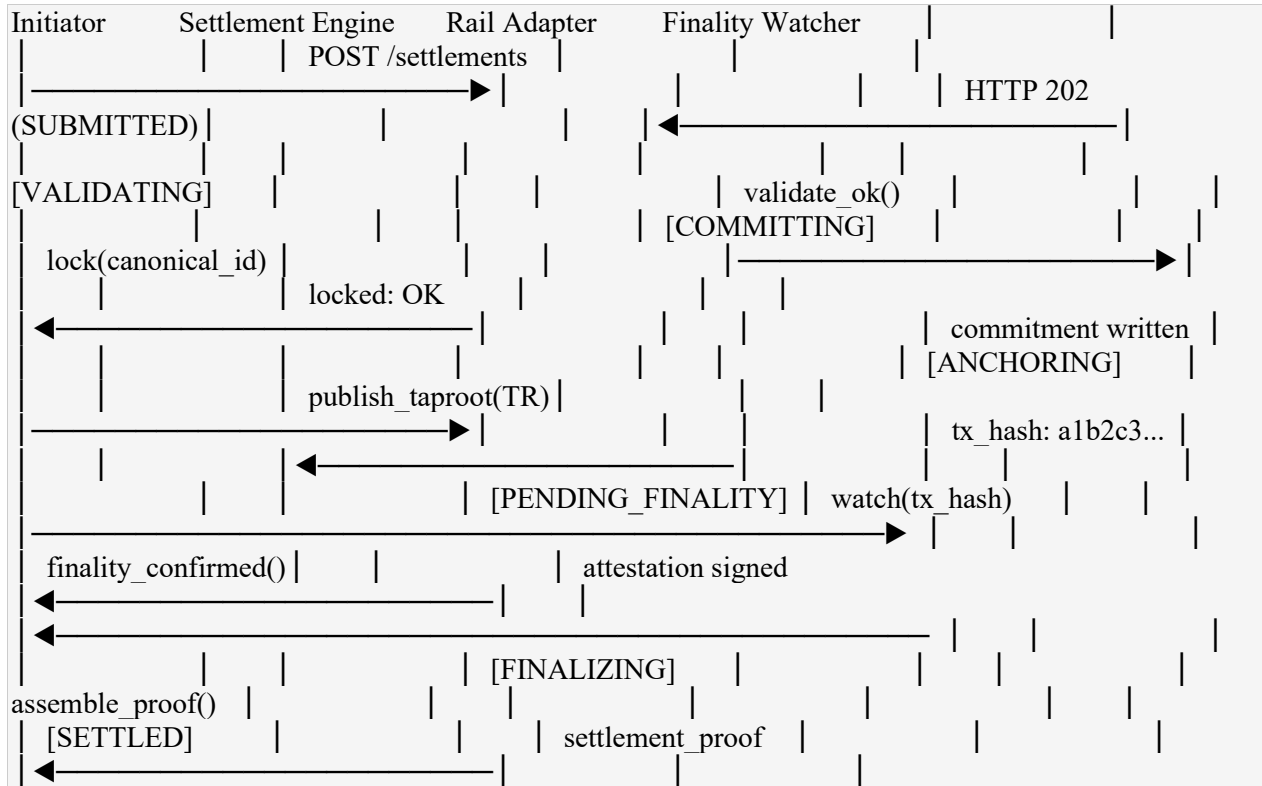
5.1.1 Description

S1 settlement confirms a canonical object state transition on a single rail. No bridging occurs. The `source_rail` and `target_rails[0]` MUST be the same `rail_id`. Despite operating on a single rail, S1 MUST still produce a Taproot Root (from a single-leaf Merkle tree), anchor it on the rail, and produce a Settlement Proof. S1 is used for any URIB canonical event that occurs within a single rail ecosystem (e.g., an NFT ownership transfer on Ethereum, an identity key update on Solana).

5.1.2 Step-by-Step Procedure

7. Initiator constructs `SettlementRequest` with `settlement_category="S1"`, `source_rail=target_rails[0]`, and `optimistic=false`.
8. Initiator signs and submits to `POST /v1/settlements`. Engine acknowledges with HTTP 202 and logs SUBMITTED.
9. Engine invokes Validators. All 7 validation checks execute (Section 8). On pass: → COMMITTING.
10. Settlement Escrow locks the canonical object on the single rail and issues one bridge commitment (`rail_from = rail_to = source_rail`). Commitment status → PENDING. Engine → ANCHORING.
11. Taproot Aggregator computes `TR = SHA3-256` of the single commitment leaf (trivial single-leaf Merkle tree). Rail Adapter publishes TR to the rail. Commitment status → ANCHORED. Engine → PENDING_FINALITY.
12. Single Finality Watcher monitors the rail until the rail's finality condition is met. Writes Finality Attestation. Commitment status → FINALIZED. Engine → FINALIZING.
13. Settlement Engine assembles Settlement Proof. All witness signatures collected. Commitment status → SETTLED. Engine → SETTLED.

5.1.3 ASCII Sequence Diagram



5.1.4 Worked Example: ERC-20 Transfer on Ethereum

Canonical Object (before):

```
{ "canonical_id": "c-hash:sha3-256:9f3a1c7e2b4d5f8a...", "semantic_id":
"sid:urib:v1:token:fungible:erc20", "state": { "owner": "0xDeadBeef...aAbBcC", "amount":
"1000000000000000000", // 1.0 ETH-denominated ERC-20 "token":
"0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48" // USDC }, "ordinal": { "rail_id":
"ethereum:mainnet", "epoch": 194, "sequence": 10041 } }
```

Canonical Object (after settlement):

```
{ "canonical_id": "c-hash:sha3-256:9f3a1c7e2b4d5f8a...", // same object "state": { "owner":
"0xCafeBabe...dDeEfF", // new owner "amount": "1000000000000000000", "token":
"0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48" }, "ordinal": { "rail_id": "ethereum:mainnet",
"epoch": 194, "sequence": 10042 } }
```

Settlement Proof (abbreviated):

```
{ "proof_id": "sp:urib:v1:c7e9a3f1b2d5...", "settlement_id": "3f7a9c2e-1b4d-4e8f-a0c5-
d2e6f3a1b7c9", "settlement_category": "S1", "taproot_root": "9a2c7e5f3b1d8a4c...",
"commitment_leaves": ["e5d3f1a7c2b9e4f6..."], // single leaf "rail_anchors": { "ethereum:mainnet":
{ "block_height": 22847391, "tx_hash": "0xf9e8d7c6b5a4f3e2...", "finality_status":
"FINALIZED", "confirmation_depth": 2 } }, "settled_at": "2026-07-09T02:13:45.000Z",
"proof_hash": "7f3a1c9e2b5d8f4a..." }
```

5.2 S2 — Cross-Rail Bilateral Settlement Flow

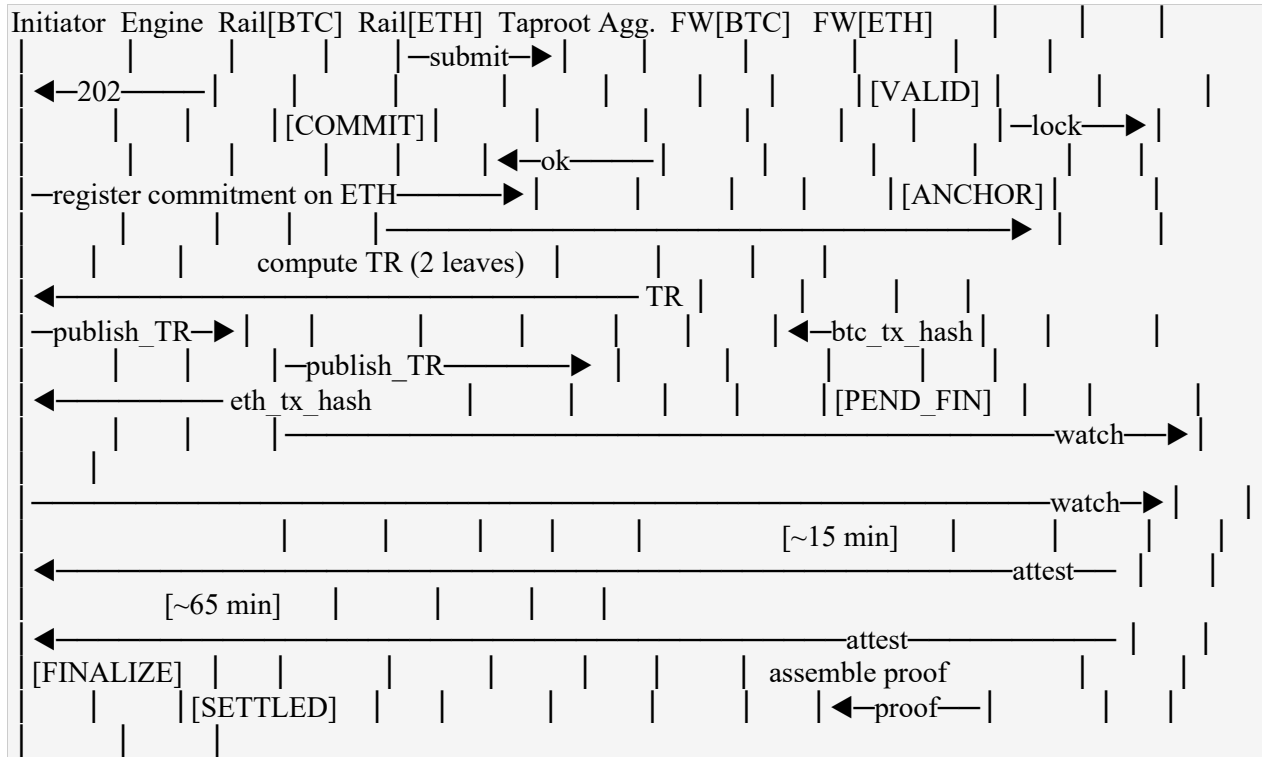
5.2.1 Description

S2 settlement bridges a canonical object between exactly two distinct rails. It requires dual-finality: both the source and destination rails MUST confirm finality before SETTLED is reached. S2 is the most common production settlement category. The Taproot Root is computed from two commitment leaves and published on both rails. The settlement is atomic: if either rail fails to finalize within the `finality_timeout`, the settlement reverts.

5.2.2 Step-by-Step Procedure

14. Initiator constructs SettlementRequest: `settlement_category="S2"`, `source_rail="bitcoin:mainnet"`, `target_rails=["ethereum:mainnet"]`.
15. Submit to engine → SUBMITTED → VALIDATING. All 7 checks run. On pass → COMMITTING.
16. Settlement Escrow locks canonical object on Bitcoin via Bitcoin Rail Adapter (OP_RETURN inscription). Issues two bridge commitment records: `commitment_leaf_1` (BTC→ETH) and `commitment_leaf_2` (ETH receipt acknowledgment). → ANCHORING.
17. Taproot Aggregator collects both leaf hashes, sorts lexicographically, computes 2-leaf Merkle tree: `TR = MerkleRoot([leaf_1, leaf_2])`. Publishes TR to Bitcoin Rail Adapter (OP_RETURN) and Ethereum Rail Adapter (URIB settlement contract event). Both commitment records → ANCHORED. → PENDING_FINALITY.
18. Two Finality Watchers activate concurrently: FW-BTC monitors Bitcoin for 6-block confirmation (~60 min); FW-ETH monitors Ethereum for epoch finalization (~12.8 min). Engine waits for BOTH.
19. FW-ETH confirms first (~15 min). `Attestation_ETH` written. FW-BTC confirms at ~65 min. `Attestation_BTC` written. Both commitments → FINALIZED. → FINALIZING.
20. Proof assembled with both `rail_anchors` and two `commitment_leaves`. → SETTLED. Total elapsed: ~67 minutes.

5.2.3 ASCII Sequence Diagram



5.2.4 Worked Example: Bitcoin UTXO Bridged to Ethereum ERC-20

Realistic Data Trace:

```

// T+0:00 — Settlement Request Submitted settlement_id: "b7e3c1a9-4f2d-4e8a-b0c6-d3f7a2e9c1b4"
canonical_id: "c-hash:sha3-256:e7a3c1f9b2d4e6a8..." source_rail: "bitcoin:mainnet" target_rails:
["ethereum:mainnet"] ordinal: { rail_id: "bitcoin:mainnet", epoch: 842, sequence: 50017 }
payload_hash: "f4c1e9a3b7d2f6e8..." // T+0:02 — Validation Passed // T+0:05 — Commitment
Issued commitment_id: "bc:urib:v1:9c3f7a1e2b4d..." state_hash_from: "7c4e1a9f2b8d3e5c..." // BTC
UTXO: outpoint a1b2c3:0, value 0.5 BTC state_hash_to: "f2a8c3e7d1b9f4a6..." // ETH: 0.5 WBTC
minted to 0xCafeBabe... // T+0:10 — Taproot Anchoring taproot_root: "9a2c7e5f3b1d8a4c..."
leaf_1_hash: "e5d3f1a7c2b9e4f6..." leaf_2_hash: "b7a4c2e9f1d3a8c5..." // BTC anchoring tx:
btc_tx_hash: "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2"
btc_block_height: 850415 (anchor tx mined) // ETH anchoring tx: eth_tx_hash:
"0xf9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0..." eth_block_height: 22847380 (anchor tx mined) //
T+0:15 — Ethereum Finality Confirmed (2 epochs = ~12.8 min) eth_confirmation_depth: 2 epochs
attestation_ETH: { block_height: 22847395, attested_at: "2026-07-09T02:13:00Z" }
commitment_status: ANCHORED → PROVEN (ETH) // T+1:05 — Bitcoin Finality Confirmed (6
blocks = ~60 min) btc_confirmation_depth: 6 attestation_BTC: { block_height: 850421, attested_at:
"2026-07-09T03:03:00Z" } commitment_status: PROVEN → FINALIZED (both rails) // T+1:06 —
FINALIZING → SETTLED settled_at: "2026-07-09T03:04:22.789Z" proof_hash:
"7f3a1c9e2b5d8f4a..."
    
```


5.3 S3 — Multi-Rail Fan-Out Settlement Flow

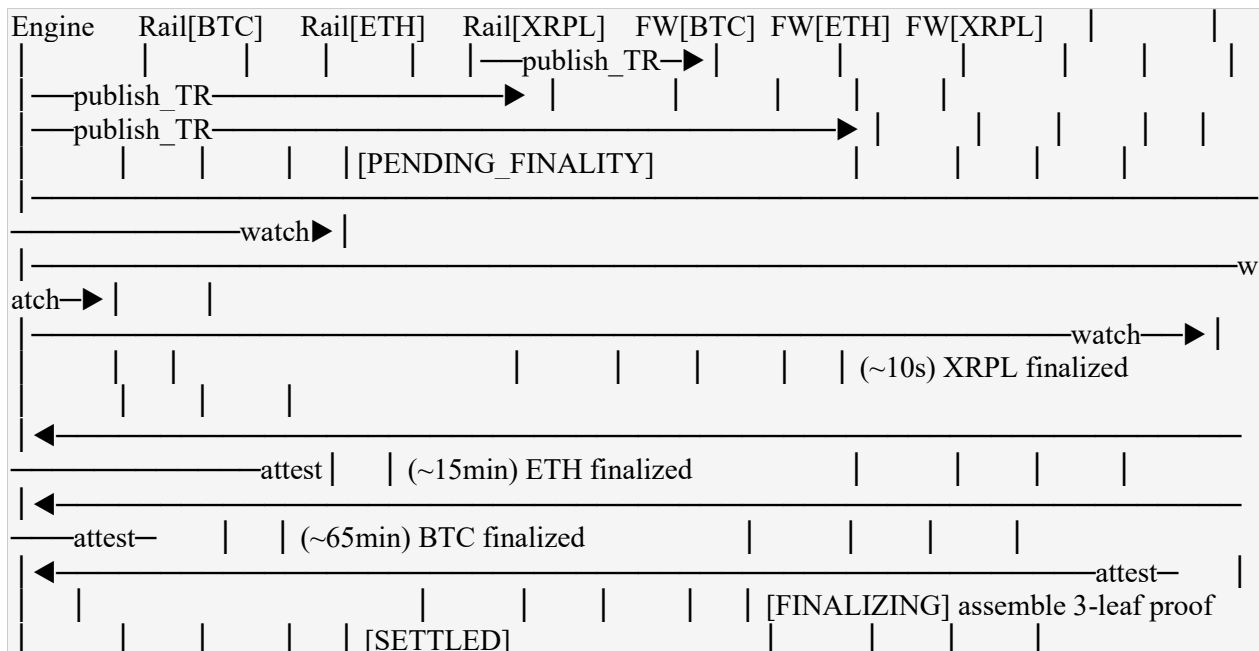
5.3.1 Description

S3 settles a canonical object simultaneously across $N \geq 3$ rails. The Taproot tree contains N leaves, and the TR is published on all N rails. All N Finality Watchers run concurrently. The slowest rail determines total settlement latency. S3 is used for global identity propagation, cross-ecosystem token listings, and multi-chain state synchronization events. The parallelism strategy is: all Rail Adapters publish the TR concurrently in phase ANCHORING; all Finality Watchers activate concurrently at PENDING_FINALITY; the engine transitions to FINALIZING only when the last Finality Watcher attests.

5.3.2 Step-by-Step Procedure

21. Initiator submits S3 SettlementRequest with
`target_rails=["bitcoin:mainnet","ethereum:mainnet","xrpl:mainnet"]`.
22. Validate (all 7 checks); confirm all 3 Rail Adapters are active. → COMMITTING.
23. Issue 3 bridge commitments (one leaf per rail). Lock canonical object on source rail. → ANCHORING.
24. Taproot Aggregator collects 3 leaf hashes, sorts lexicographically, computes 3-leaf Merkle tree.
`TR = MerkleRoot([L1, L2, L3])`. Publishes TR concurrently to all 3 Rail Adapters. → PENDING_FINALITY.
25. Three Finality Watchers activate concurrently: FW-XRPL (~10s), FW-ETH (~15 min), FW-BTC (~65 min). XRPL and ETH confirm early; settlement waits for BTC. → FINALIZING when BTC confirms.
26. Proof assembled with 3 rail_anchors and 3 commitment_leaves. → SETTLED. Elapsed: ~67 minutes (bottlenecked by Bitcoin).

5.3.3 ASCII Sequence Diagram (3 Rails in Parallel)



5.3.4 Taproot Tree (3-Leaf Example)

Commitment leaves (lexicographically sorted by commitment_id): L1 = SHA3-256(leaf_content_BTC) = "b1c2d3e4f5a6b7c8..." L2 = SHA3-256(leaf_content_ETH) = "c2d3e4f5a6b7c8d9..." L3 = SHA3-256(leaf_content_XRPL) = "d3e4f5a6b7c8d9e0..." Merkle Tree Construction (balanced, odd leaf duplicated): Level 1 (leaves): [L1, L2, L3, L3] Level 2 (pairs): [H(L1,L2), H(L3,L3)] Level 3 (root): [H(H(L1,L2), H(L3,L3))] where H(a,b) = SHA3-256(a || b) with a,b sorted lexicographically Taproot Root (TR) = "9a2c7e5f3b1d8a4c..." Merkle path for L1: [{ sibling: L2, position: "right" }, { sibling: H(L3,L3), position: "right" }] Merkle path for L3: [{ sibling: L3, position: "right" }, { sibling: H(L1,L2), position: "left" }]

5.4 S4 — Optimistic Settlement Flow

5.4.1 Description

S4 enables optimistic settlement by provisionally accepting settlement after anchoring is complete (or after the first rail confirms), without waiting for all rail finalities. A challenge window opens during which any party can dispute the settlement by submitting a counter-proof and a challenge bond. If no valid challenge is received before the window expires, the settlement is finalized. This dramatically reduces perceived latency for use cases that can tolerate a time-bounded finality risk.

5.4.2 Challenge Window, Bond, and Submission Format

- **Challenge Window:** Opens at OPTIMISTIC state entry. Duration = `challenge_window_seconds` (default 3600s, range 600–86400s). Exact expiry = `now_at_OPTIMISTIC_entry` + `challenge_window_seconds`.
- **Challenge Bond:** Challenger MUST lock a bond in the Settlement Escrow prior to challenge submission. Minimum bond: `max(settlement_floor_bond, 1% of settlement_value)`. Bond held until arbitration resolution. If challenger wins: bond returned. If challenger loses: bond slashed and transferred to Initiator and protocol treasury (50/50 split).
- **Challenge Submission Format:** See `DisputeRecord.counter_proof` schema in Section 3.5. Challenger MUST provide: `proof_type`, `claimed_state_hash`, `actual_state_hash`, `evidence_tx_hash`, `evidence_rail`, `challenger_sig`.

5.4.3 Timing Diagram

Timeline: T+0:00 ANCHORING complete → OPTIMISTIC state entered |—— Challenge Window Opens | [challenge_window_seconds = 3600s] T+0:15 ETH Finality confirmed (non-optimistic path would wait here) | In S4, settlement already OPTIMISTIC — no state change yet | T+1:00 [Challenge Window Expires — no challenge received] | T+1:00 → FINALIZING (optimistic finalization) T+1:01 → SETTLED |—— Total elapsed: ~61 minutes vs ~67 min for S2 (minor saving here) For Solana-to-Ethereum: ~15s OPTIMISTIC vs ~15min full S2 High-value difference on fast source rail + slow destination rail ——— If challenged at T+0:30: —————

T+0:30 Challenge received → CHALLENGED → ARBITRATION T+14:30 Arbitration resolves (up to 24hr window) → ARBITRATION_RESOLVED_VALID → resume PENDING_FINALITY → SETTLED OR → ARBITRATION_RESOLVED_INVALID → REVERTING → REVERTED

5.4.4 Worked Example: Solana-to-Ethereum Optimistic Bridge (No Challenge)

```
// S4 Settlement with optimistic=true, challenge_window_seconds=3600 T+0:00 Submit.
settlement_category="S4", source_rail="solana:mainnet", target_rails=["ethereum:mainnet"] T+0:02
Validation passed T+0:05 Commitment issued; Solana account state locked T+0:18 TR anchored on
Solana (slot 285,711,342) and Ethereum (block 22,847,410) T+0:19 Solana finality confirmed (32-slot
confirmation = ~15s) → OPTIMISTIC state entered → challenge_window_expires_at =
T+1:00:19 T+1:00 ETH finality confirmed (independently; no effect on S4 flow) T+1:00:19 Challenge
window expires — no challenge received T+1:00:20 → FINALIZING; proof assembled T+1:00:22 →
SETTLED Total perceived latency: ~60 minutes (challenge window dominated) Effective
latency if challenge_window_seconds=600: ~12 minutes
```

5.5 S5 — Identity Continuity Settlement Flow

5.5.1 Description

S5 handles ThreadZero lifecycle events: genesis (identity creation), key rotation, fork (identity branching into two identities), and tombstone (permanent identity deactivation). S5 MUST settle on every rail where the identity is currently active. Partial finality is NEVER permitted for S5. The Settlement Proof MUST include an `identity_continuity_proof` that traces the unbroken chain of ThreadZero events from genesis to the current event. S5 is the most security-critical settlement category.

5.5.2 Step-by-Step Procedure (Key Rotation Example)

27. ThreadZero event generated by identity owner: `event_type="KEY_ROTATION"`, new public key provided, signed by current (outgoing) key.
28. Settlement Engine validates thread integrity: verifies the ThreadZero chain from genesis to the rotation event, confirms no fork anomaly, confirms the signing key matches the current active key registered on all rails.
29. Issue N bridge commitments (one per active rail). Lock identity state on all rails simultaneously.
→ ANCHORING.
30. Taproot Aggregator computes TR from N commitment leaves. Publish TR to all N rails concurrently. → PENDING_FINALITY.
31. All N Finality Watchers activate. MUST wait for ALL rails to finalize. → FINALIZING.
32. Assemble Settlement Proof including `identity_continuity_proof` (full chain from genesis). → SETTLED. New key is now authoritative on all rails.

5.5.3 Identity Continuity Proof (Key Rotation, 3 Rails)

```
// identity_continuity_proof structure (within SettlementProof) { "thread_id":  
"thread:urib:z6Mkf5r...Qvc", "genesis_event": { "event_type": "GENESIS", "event_hash":  
"a1b2c3d4e5f6...", "settled_at": "2024-03-15T10:00:00Z", "settlement_id": "aabb1122-...",  
"active_rails": ["bitcoin:mainnet"] }, "event_chain": [ { "event_type":  
"RAIL_REGISTRATION", "event_hash": "b2c3d4e5f6a7...", "settled_at": "2024-06-  
01T09:00:00Z", "new_rail": "ethereum:mainnet", "active_rails":["bitcoin:mainnet",  
"ethereum:mainnet"] }, { "event_type": "RAIL_REGISTRATION", "event_hash":  
"c3d4e5f6a7b8...", "settled_at": "2025-01-10T14:00:00Z", "new_rail": "solana:mainnet",  
"active_rails":["bitcoin:mainnet","ethereum:mainnet","solana:mainnet"] }, { "event_type":  
"KEY_ROTATION", "event_hash": "d4e5f6a7b8c9...", // current event being settled  
"prev_key": "Ed25519:z6Mkf5r...Qvc", "new_key": "Ed25519:z6Mkp9r...Xyz",  
"rotation_sig": "base64url:MEUCIQDx...", // signed by prev_key "settled_at": "2026-07-  
09T02:10:22Z", "active_rails": ["bitcoin:mainnet","ethereum:mainnet","solana:mainnet"] } ],  
"chain_hash": "e5f6a7b8c9d0...", // SHA3-256 of the entire event_chain array, proving continuity  
"oracle_sig": "base64url:MEQCIAX..." }
```

5.6 S6 — Disputed Settlement Flow

5.6.1 Description

S6 covers the full dispute lifecycle from CHALLENGED through ARBITRATION to resolution. Disputes arise when a party believes a settlement's commitment contains a false state claim. The challenger MUST post a bond and submit a counter-proof. The Dispute Arbitrator evaluates both the original commitment and the counter-proof and issues a binding, signed resolution.

5.6.2 Arbitration Procedure

33. Challenger submits to POST /v1/settlements/{id}/challenge with bond tx and counter-proof.
Engine validates challenge submission (structural validity, bond confirmation). →
CHALLENGED → ARBITRATION.
34. Arbitrator retrieves: original SettlementRequest, BridgeCommitment, FinalityAttestations, and challenger's CounterProof.
35. Arbitrator executes deterministic resolution algorithm:
 - Re-derive the expected state_hash_to from the canonical object and the target rail state at the block/ledger specified in the commitment.
 - Compare against the state_hash_to in the commitment.
 - If match: resolution = VALID. If mismatch: resolution = INVALID.

36. Arbitrator signs resolution with `arbitrator_sig` and publishes on-chain (preferred) or to the dispute log.
37. Bond slashing / return executes per Section 5.4.2.
38. Settlement resumes (VALID) or reverts (INVALID).

5.6.3 Worked Example: Challenged Bitcoin-to-Ethereum Bridge

```
// SCENARIO: Initiator submitted incorrect state_hash_to in commitment. // Actual ETH WBTC mint
amount was 0.499 BTC (fee deducted), // but commitment claimed 0.500 BTC. Challenger detects this
on-chain. T+0:00 S4 settlement OPTIMISTIC (BTC→ETH, challenge_window=3600s)
commitment.state_hash_to = SHA3-256("0.500 WBTC to 0xCafe...")      Actual on-chain: 0.499
WBTC minted (fee applied) T+0:45 Challenger submits:      counter_proof.proof_type =
"state_hash_mismatch"      counter_proof.claimed_hash = commitment.state_hash_to // wrong value
counter_proof.actual_hash = SHA3-256("0.499 WBTC to 0xCafe...")      counter_proof.evidence_tx
= "0xb8a7c6e5d4f3b2a1..." // ETH tx      counter_proof.evidence_rail = "ethereum:mainnet"
challenge_bond_tx          = "0xc9d0e1f2a3b4c5d6..." // bond posted T+0:46 → CHALLENGED →
ARBITRATION      DisputeRecord created: "dr:urib:v1:2c4e6a8f..." T+8:30 Arbitrator re-derives
expected state_hash_to:      - Queries eth block 22847395 for WBTC mint event      - Actual mint:
0.499 WBTC (fee deducted per WBTC protocol)      - actual_state_hash = SHA3-256("0.499 WBTC to
0xCafe...")      - actual_state_hash != commitment.state_hash_to      - Resolution: INVALID T+8:31
→ ARBITRATION_RESOLVED_INVALID → REVERTING      - Bitcoin UTXO unlocked (URIB
reverts lock; UTXO returned to Initiator)      - Ethereum WBTC mint reversed via URIB settlement
contract      - Nullifier issued for ordinal { epoch:842, seq:50017 }      - Challenger bond
RETURNED (challenger was correct)      - Initiator penalized: protocol penalty fee applied T+8:45
→ REVERTED      settlement_status = REVERTED      error_code =
"E401_ARBITRATION_INVALID"      Initiator notified: "Re-submit with correct state_hash_to"
```

6. Rail-Specific Settlement Paths

6.1 Bitcoin Settlement Path

Finality Model: Nakamoto consensus. URIB requires **6 block confirmations** for FINALIZED status (~60 minutes at ~10 min/block average). Fewer confirmations are permitted only in test networks. URIB MUST NOT accept fewer than 3 confirmations under any production configuration.

UTXO Locking: The Bitcoin Rail Adapter locks a canonical object by committing an `OP_RETURN` output encoding the `settlement_id` and `canonical_id` in a specially crafted UTXO. The UTXO is unspendable (`OP_RETURN`), forming a permanent commitment record on Bitcoin.

Taproot Root Anchoring: The TR is published to Bitcoin via one of two methods: (1) OP_RETURN output (80-byte max; TR as 32-byte SHA3-256 hash encoded as hex in the output) or (2) Taproot script-path commitment in a P2TR output. Method (2) is preferred for richer proof structures. The anchoring tx MUST include the `settlement_id` in OP_RETURN for indexability.

Settlement Latency: Expected ~65 minutes from ANCHORING to FINALIZED. Engineers MUST set `max_finality_wait_seconds` ≥ 4800 (80 min) for Bitcoin-involved settlements.

Error Conditions:

- **Orphaned Block (E401):** Anchor block not in the longest chain. Finality Watcher detects reorg. Re-anchor required (see Section 11.4).
- **RBF Replacement (E304):** Anchoring tx replaced by a higher-fee tx before mining. Rail Adapter MUST monitor mempool and detect RBF replacements. Re-submit anchor tx with higher fee.
- **Mempool Eviction (E304):** Anchoring tx evicted from mempool due to low fee. Rail Adapter MUST monitor tx status and re-broadcast with bumped fee if not mined within 20 minutes.

6.2 Ethereum Settlement Path

Finality Model: Post-Merge Gasper consensus. URIB requires **epoch finalization** (2 epochs, ~12.8 minutes) for FINALIZED. Slot-level confirmation (~12s) is NOT sufficient for PROVEN; epoch finalization is required for FINALIZED. URIB MAY use slot confirmation for OPTIMISTIC (S4) settlements on Ethereum.

State Commitment: The Ethereum Rail Adapter submits a transaction to the URIB Settlement Contract deployed on Ethereum. The contract emits a `CommitmentAnchored(settlement_id, canonical_id, taproot_root, timestamp)` event log. This event is the on-chain anchor record. The event log is indexed by the Finality Watcher.

Gas Estimation: Rail Adapter MUST query gas price via EIP-1559 base fee + priority fee estimate. MUST apply a 1.2x multiplier to the gas estimate for safety. If initial tx fails due to gas underestimation, Rail Adapter MUST resubmit with 1.5x original gas limit. Max 3 gas-related retries (E303).

Settlement Latency: Expected ~15 minutes from ANCHORING to FINALIZED (epoch finalization).

Reorg Handling (E401): Ethereum reorgs deeper than 2 blocks are rare post-Merge but possible. Finality Watcher monitors for reorgs by checking that the anchor tx's block_hash remains in the canonical chain. If a reorg is detected and the anchor tx is excluded, re-anchor is required (Section 11.4).

6.3 XRPL Settlement Path

Finality Model: XRP Ledger uses Byzantine Fault Tolerant consensus. Ledger close time ~3–5 seconds. URIB requires **2 ledger confirmations** (confirmed + validated) for FINALIZED. Validated ledger close is final; no reorgs occur on XRPL after validation.

Commitment Method: URIB anchors on XRPL via one of: (1) Memo field on an AccountSet transaction from the URIB XRPL account (encodes settlement_id and taproot_root as hex in Memo data), or (2) NFToken mint with settlement metadata. Method (1) is preferred for speed and cost efficiency.

Settlement Latency: Expected ~10 seconds from ANCHORING to FINALIZED. The fastest URIB-supported rail.

Error Conditions:

- **Ledger Gap (E402):** Disconnected from XRPL network during ledger close. Finality Watcher reconnects and queries ledger index to confirm anchor.
- **Validator Disagreement:** If XRPL validator quorum fails (<80% agreement), ledger close delayed. Rail Adapter MUST timeout after 30 seconds and retry.

6.4 Solana Settlement Path

Finality Model: Solana uses Tower BFT with ~0.4s slot time. URIB uses **optimistic finality at 32-slot confirmation** (~15 seconds) for PROVEN status, and **root commitment** (approximately 31-slot supermajority) for FINALIZED. For S4 (optimistic) settlements, URIB accepts 1-slot confirmation for OPTIMISTIC state entry on Solana.

Commitment Method: URIB writes commitment to a dedicated Solana program account using the URIB Settlement Program (a Solana on-chain program). The program account stores the `settlement_id`, `taproot_root`, and `canonical_id`. Program instruction: `anchor_commitment(settlement_id, taproot_root, canonical_id)`.

Settlement Latency: Expected ~15 seconds for FINALIZED. Solana is the second-fastest URIB-supported rail (after XRPL).

Error Conditions:

- **Slot Skip (E402):** Leader fails to produce a block for a slot. Finality Watcher accounts for slot skips in confirmation depth calculation.
- **Leader Failure:** Consecutive leader failures can delay anchoring by a few seconds. Rail Adapter MUST retry anchor submission in the next slot.
- **Network Partition:** Solana has historically experienced full outages. If Rail Adapter cannot reach Solana RPC within 30 seconds, emit E302.

6.5 IPFS / Content-Addressed Rail Settlement Path

Finality Model: IPFS is content-addressed; the CID of a pinned object is its final address. URIB requires the canonical object's CID to be pinned on **at least 3 independent pinning nodes** for FINALIZED.

Pinning SLA (time to pin on 3 nodes) is provider-dependent, typically 5–30 seconds.

Settlement Method: The canonical object's state (or state delta) is serialized as a URIB-format DAG-CBOR document, pinned to IPFS, and its CID is embedded in the bridge commitment as the `state_hash_to`. The Taproot Root is published to IPFS as a signed URIB anchor document pinned alongside the canonical object.

Settlement Latency: Expected 5–60 seconds depending on pinning SLA. URIB recommends using Pinata, web3.storage, or equivalent enterprise pinning services with SLA guarantees.

6.6 HTTPS / Off-Chain API Rail Settlement Path

Finality Model: An HTTPS rail is any off-chain API endpoint registered as a URIB rail. Finality = HTTP 200 response with a signed acknowledgment from the recipient service. URIB requires the acknowledgment to be signed with the recipient's registered Ed25519 key.

Commitment Method: URIB delivers a HMAC-SHA256-signed payload (using the `settlement_escrow` HMAC key) to the recipient API. Payload includes `ordinal`, `settlement_id`, `canonical_id`, `state_hash_to`, and `taproot_root`. Recipient MUST return: `{ "ack_status": "RECEIVED", "recipient_sig": "base64url:..." }`.

Settlement Latency: Near-instant (<1 second round trip for co-located services; <5 seconds for remote).

Replay Protection: The payload MUST include a nonce (UUID v4) and a timestamp. The recipient MUST reject any payload with a timestamp older than 60 seconds or a previously seen nonce (nonce deduplication window: 24 hours).

6.7 Rail Comparison Table

Rail	Finality Model	URIB Finality Requirement	Expected Settlement Latency	Commitment Method	Reorg Risk
Bitcoin	Nakamoto Longest Chain	6 block confirmations	~65 minutes	OP_RETURN or P2TR	Medium (orphan blocks possible)
Ethereum	Gaspar (LMD-GHOST + Casper FFG)	Epoch finalization (2 epochs)	~15 minutes	URIB Settlement Contract event log	Low (reorgs >2 blocks rare post-Merge)
XRPL	BFT Consensus (UNL)	2 validated ledger closes	~10 seconds	AccountSet Memo or NFToken	None (validated ledger is final)
Solana	Tower BFT	32-slot confirmation (PROVEN); root commitment (FINALIZED)	~15 seconds	URIB Settlement Program account	Very Low (leader-rotation based)
IPFS	Content-addressed (CID pinned)	Pinned on ≥ 3 independent nodes	5–60 seconds	CID in bridge commitment	None (content-addressed)
HTTPS API	Signed HTTP acknowledgment	HTTP 200 + recipient_sig	<5 seconds	HMAC-signed payload delivery	None (off-chain)

7. Timing, Latency, and Timeout Model

7.1 Settlement Latency Decomposition

Total settlement latency is decomposed into the following components. All durations are measured from the Settlement Engine's perspective.

Component	Symbol	Description	Typical Range
Validation Time	T_{validate}	Time to execute all 7 validation checks including Ordinal Oracle query and signature verification.	50ms – 2s
Commitment Time	T_{commit}	Time to issue and write all bridge commitments and lock canonical object on source rail.	100ms – 5s
Taproot Computation	T_{taproot}	Time to compute the Merkle tree and Taproot Root. $O(N \log N)$ for N leaves.	<10ms
Anchoring per Rail	$T_{\text{anchor}[i]}$	Time to publish TR on rail i (tx submission + first confirmation).	10s (XRPL) – 600s (BTC)
Finality per Rail	$T_{\text{finality}[i]}$	Time for rail i to reach its URIB finality requirement after anchoring.	5s (XRPL) – 3900s (BTC)
Proof Assembly	T_{assemble}	Time to collect all attestations and compute the Settlement Proof.	50ms – 500ms

Total Latency Formula:

$T_{\text{total}} = T_{\text{validate}} + T_{\text{commit}} + T_{\text{taproot}} + \max_i(T_{\text{anchor}[i]}) + \max_i(T_{\text{finality}[i]}) + T_{\text{assemble}}$
 For S4 (optimistic, no challenge): $T_{\text{total}} = T_{\text{validate}} + T_{\text{commit}} + T_{\text{taproot}} + \max_i(T_{\text{anchor}[i]}) + \text{challenge_window_seconds}$ // dominates for long windows
 For S3 (multi-rail fan-out): $T_{\text{total}} = T_{\text{validate}} + T_{\text{commit}} + T_{\text{taproot}} + \max_i(T_{\text{anchor}[i]})$ // parallel anchoring; max determines
 parallel watching; slowest rail determines + $\max_i(T_{\text{finality}[i]})$ // + T_{assemble}

7.2 Timeout Hierarchy

Stage	Timeout Parameter	Default Value	Behavior on Timeout
Validation	validation_timeout	30 seconds	→ VALIDATION_FAILED (E106 ExpiredRequest)
Commitment	commitment_timeout	60 seconds per attempt	→ COMMITMENT_FAILED; retry up to 3 times with exponential backoff
Anchoring (per rail)	anchoring_timeout[rail]	120 seconds	→ ANCHORING_FAILED (E300); retry up to 3 times
Finality Watch — Bitcoin	finality_timeout[bitcoin]	7200 seconds (2 hours)	→ EXPIRED (E400 FinalityTimeout)
Finality Watch — Ethereum	finality_timeout[ethereum]	1800 seconds (30 min)	→ EXPIRED (E400)
Finality Watch — XRPL	finality_timeout[xrpl]	60 seconds	→ EXPIRED (E400)
Finality Watch — Solana	finality_timeout[solana]	120 seconds	→ EXPIRED (E400)
Finality Watch — IPFS	finality_timeout[ipfs]	300 seconds	→ EXPIRED (E400)
Finality Watch — HTTPS	finality_timeout[https]	30 seconds	→ EXPIRED (E400)
Challenge Window	challenge_window_seconds	3600 seconds (1 hour)	→ FINALIZING (SETTLED if no challenge received)
Arbitration	arbitration_timeout	86400 seconds (24 hours)	Escalate to admin; settlement remains in ARBITRATION
Overall Settlement	expires_at (in SettlementRequest)	Caller-set; MUST be > max rail finality time	→ EXPIRED; if committed: → REVERTING first

7.3 Retry Policy

The following retry policy is normative for all retrievable stages:

- **Retriable stages:** COMMITTING (E200), ANCHORING (E300, E301, E302, E303, E304), FINALIZING (E602).
- **Non-retriable stages:** VALIDATING (all E1xx errors; errors indicate caller fault).
- **Max retries:** 3 attempts per stage (4 total including the initial attempt).
- **Backoff strategy:** Exponential backoff with jitter. Wait = $\min(\text{base} * 2^{\text{attempt}} + \text{rand}(0, \text{base}), \text{max_wait})$ where $\text{base}=2\text{s}$, $\text{max_wait}=128\text{s}$. Jitter prevents thundering herd.
- **Idempotency:** All retry attempts MUST use the identical `settlement_id`. The `settlement_id` is the global idempotency key. Retried operations MUST be idempotent: if the operation was already successfully completed in a previous attempt, the retry MUST detect and accept the existing result rather than re-executing.
- **Retry logging:** Every retry attempt MUST be logged as a `SettlementLogEntry` with metadata `retry_attempt: N`.

7.4 Latency Benchmark Table

Estimated end-to-end settlement latency by category and rail combination. Values assume no disputes, no retries, and optimal network conditions. Bitcoin-involved settlements dominate by finality time.

Category	Bitcoin	Ethereum	XRPL	Solana	IPFS	HTTPS
S1 (single rail)	~65 min	~15 min	~10 sec	~15 sec	~30 sec	<5 sec
S2 (BTC src)	~67 min	~67 min	~67 min	~67 min	~67 min	~67 min
S2 (ETH src)	~67 min	~15 min	~15 min	~15 min	~15 min	~15 min
S3 (incl. BTC)	~67 min (bottlenecked by Bitcoin in all N-rail combinations)					
S3 (ETH+XRPL+SOL)	~15 min (bottlenecked by Ethereum epoch finality)					
S4 (optimistic, 1hr window)	~62 min	~62 min	~62 min	~62 min	~62 min	~62 min
S4 (optimistic, 10min window)	~12 min	~12 min	~12 min	~12 min	~12 min	~12 min
S5 (3-rail incl. BTC)	~67 min (all active rails must finalize)					

8. Settlement Validation Procedures

All checks in this section are executed during the VALIDATING phase and MUST all pass before the settlement engine transitions to COMMITTING. Checks are executed in the order listed. A failing check MUST immediately halt validation and return the specified error code. Partial validation (proceeding despite a failed check) is a critical protocol violation.

8.1 Canonical Integrity Check

39. [CANON-1] Recompute the C-Hash from the canonical object's fields using the URIB C-Hash derivation algorithm (see Master Spec). Assert: `recomputed_c_hash == settlement_request.canonical_id`. Failure: **E100 InvalidSignature** (misuse; actually canonical_id mismatch — return specific sub-code E100.1).
40. [CANON-2] Retrieve the canonical object payload. Compute `SHA3-256(payload)`. Assert: `computed_hash == settlement_request.payload_hash`. Failure: **E100**.
41. [CANON-3] Verify the ordinal is not already recorded in the ordinal registry for this (rail_id, canonical_id) pair. Failure: **E101 StaleOrdinal**.
42. [CANON-4] Verify the ordinal sequence is strictly greater than the last known sequence for this (rail_id, canonical_id) pair. Failure: **E101 StaleOrdinal**.

8.2 Semantic Consistency Check

43. [SEM-1] Resolve the SID against the URIB SID registry. Assert: SID is registered and not deprecated. Failure: **E103 SemanticMismatch**.
44. [SEM-2] Verify the canonical object's semantic type (as defined by the SID) is permitted for settlement on each target rail. Query each Rail Adapter for its supported semantic types. Assert: `semantic_type IN rail.supported_semantic_types` for all target rails. Failure: **E103 SemanticMismatch**.
45. [SEM-3] Verify no semantic tag conflicts exist between the source canonical object and any target rail's constraints (e.g., a rail that only accepts fungible tokens rejects non-fungible semantic types). Failure: **E103 SemanticMismatch**.

8.3 Initiator Authorization Check

46. [AUTH-1] Resolve the Initiator's public key from `initiator_id`. Failure if not resolvable: **E104 UnauthorizedInitiator**.
47. [AUTH-2] Verify `initiator_sig` is a valid Ed25519 signature over SHA3-256(`settlement_id` || `canonical_id` || `ordinal.rail_id` || `ordinal.epoch` || `ordinal.sequence`) using the resolved public key. Failure: **E100 InvalidSignature**.
48. [AUTH-3] Verify the Initiator has ownership or delegation authority over the canonical object. Query the canonical object's ownership record. Failure: **E104 UnauthorizedInitiator**.

8.4 Rail Capability Check

49. [RAIL-1] Verify every `rail_id` in `target_rails` is registered in the URIB rail registry. Failure: **E105 RailUnavailable**.
50. [RAIL-2] Ping each Rail Adapter for all `target_rails`. Assert: HTTP 200 (or adapter-specific healthcheck pass) within 5 seconds. Failure: **E105 RailUnavailable**.
51. [RAIL-3] Verify each Rail Adapter reports that it can accept the canonical object type (semantic type and size). Failure: **E105 RailUnavailable**.

8.5 Bridge Commitment Pre-Check

52. [COMMIT-1] Query the commitment store: assert no active bridge commitment exists for `canonical_id` in PENDING, ANCHORED, PROVEN, or DISPUTED status (Non-Duplication Invariant). Failure: **E107 ConflictingCommitment**.
53. [COMMIT-2] Query the settlement store: assert no PENDING, VALIDATING, COMMITTING, ANCHORING, PENDING_FINALITY, OPTIMISTIC, CHALLENGED, or ARBITRATION settlement exists for `canonical_id`. Failure: **E102 DuplicateSettlement**.
54. [COMMIT-3] Query the nullifier registry: assert the ordinal has not been nullified. Failure: **E101 StaleOrdinal**.

8.6 Ordinal Freshness Check

55. [ORD-1] Assert: $\text{ordinal.epoch} \geq \text{current_epoch} - \text{max_epoch_lag}$ where $\text{max_epoch_lag} = 10$ by default. Stale epochs indicate replayed old requests. Failure: **E101 StaleOrdinal**.
56. [ORD-2] Assert: $\text{ordinal.sequence} > \text{last_known_sequence}(\text{rail_id}, \text{canonical_id})$. Failure: **E101 StaleOrdinal**.
57. [ORD-3] Verify the ordinal has been issued by the Ordinal Oracle (verify Ordinal Oracle's signature on the ordinal proof). Failure: **E101 StaleOrdinal**.

8.7 Finality Window Compatibility Check

58. [FIN-1] If $\text{optimistic}=\text{false}$: compute the maximum expected settlement time as $T_{\text{validate}} + T_{\text{commit}} + \max_i(T_{\text{anchor}[i]}) + \max_i(T_{\text{finality}[i]}) + T_{\text{assemble}}$. Assert: $\text{expires_at} \geq \text{now}() + \text{max_expected_settlement_time} + 300\text{s buffer}$. Failure: **E106 ExpiredRequest**.
59. [FIN-2] If $\text{optimistic}=\text{true}$: assert $\text{challenge_window_seconds} \geq 600$. Failure: validation error (bad request).
60. [FIN-3] Assert: $\text{expires_at} > \text{now}() + 60\text{s}$ (minimum future time). Failure: **E106 ExpiredRequest**.

Summary: Validation Checks and Error Codes

Check ID	Check Name	Error Code on Failure	Retriable
CANON-1	C-Hash recomputation	E100	No
CANON-2	Payload hash match	E100	No
CANON-3	Ordinal not in registry	E101	No
CANON-4	Ordinal sequence monotonicity	E101	No
SEM-1	SID registered	E103	No
SEM-2	Semantic type $\times$ rail compatibility	E103	No
SEM-3	No semantic tag conflict	E103	No
AUTH-1	Initiator key resolution	E104	No
AUTH-2	initiator_sig verification	E100	No

Check ID	Check Name	Error Code on Failure	Retriable
AUTH-3	Ownership or delegation	E104	No
RAIL-1	Rail registry lookup	E105	Yes (transient)
RAIL-2	Rail Adapter liveness ping	E105	Yes (transient)
RAIL-3	Rail accepts canonical type	E105	No
COMMIT-1	Non-Duplication Invariant	E107	No
COMMIT-2	No duplicate active settlement	E102	No
COMMIT-3	Nullifier check	E101	No
ORD-1	Epoch freshness	E101	No
ORD-2	Sequence monotonicity	E101	No
ORD-3	Oracle signature on ordinal	E101	No
FIN-1	expires_at compatibility	E106	No
FIN-2	challenge_window minimum	E106	No
FIN-3	expires_at minimum future	E106	No

9. Bridge Commitment Lifecycle (Settlement Context)

9.1 Commitment Issuance

Bridge commitments are issued by the Settlement Escrow during the COMMITTING phase. The Settlement Escrow acts on behalf of the Initiator but does NOT require the Initiator to be online during issuance. Commitment issuance is an atomic operation: all commitment records for a given settlement_id MUST be written in a single atomic database transaction. If any single write fails, all writes are rolled back and the settlement transitions to COMMITMENT_FAILED.

Each commitment record is signed by the Settlement Escrow key at issuance time: `escrow_sig = Ed25519_sign(escrow_key, SHA3-256(commitment_id || canonical_id || state_hash_from || state_hash_to || ordinal_from || ordinal_to))`.

9.2 Commitment Anchoring

During the ANCHORING phase, the Taproot Aggregator performs the following steps:

61. Retrieve all bridge commitment records for `settlement_id`.
62. Extract `taproot_leaf_hash` from each commitment: `leaf_hash_i = SHA3-256(commitment_id_i || state_hash_from_i || state_hash_to_i || ordinal_from_i || ordinal_to_i)`.
63. Sort leaf hashes lexicographically.
64. Compute Merkle Tree: `TR = MerkleRoot(sorted_leaf_hashes)`. See Appendix C for reference implementation.
65. Submit TR to each Rail Adapter concurrently via `publish_taproot(rail_id, TR, settlement_id)`.
66. On success from all Rail Adapters: write `taproot_root` to all commitment records; update `commitment_status` from PENDING to ANCHORED.

9.3 Proof Generation

A commitment proof is generated by computing the Merkle authentication path from a specific commitment leaf to the Taproot Root. Any party can independently verify the proof by:

67. Recomputing the `leaf_hash` from the commitment's public fields.
68. Walking the Merkle path: at each step, hash the current node with the sibling (sorted order) to produce the parent node.
69. Asserting the final result equals the Taproot Root published on-chain.

On successful on-chain verification: `commitment_status` → PROVEN.

9.4 Finality Confirmation

Finality Watchers poll each rail according to the poll schedule:

- **Bitcoin:** Poll every 60 seconds; check confirmation depth of anchor tx.
- **Ethereum:** Subscribe to beacon chain finality events; trigger on epoch finalization containing anchor block.
- **XRPL:** Subscribe to `ledger_closed` events; confirm 2 validated ledger closures after anchor ledger.
- **Solana:** Subscribe to `slotUpdate` events; confirm 32-slot depth.
- **IPFS:** Poll pinning service API every 10 seconds; confirm pin on ≥ 3 nodes.
- **HTTPS:** Await `recipient_sig` in response; immediate confirmation.

On finality confirmation: Finality Watcher writes signed `FinalityAttestation` to commitment record.

`commitment_status` → `FINALIZED`. If ALL commitment records for `settlement_id` reach `FINALIZED`:

Settlement Engine transitions to `FINALIZING`. 9.5 Settlement Completion

The Settlement Engine assembles the Settlement Proof when all commitments are `FINALIZED`. The assembly process:

70. Collect all `FinalityAttestations` for all commitment records.
71. Collect `ordinal_proof` from Ordinal Oracle.
72. Assemble `rail_anchors` map from `FinalityAttestations`.
73. Compute `merkle_path` for the primary commitment leaf.
74. If S5: attach `identity_continuity_proof`.
75. Collect `witness_sigs` from Settlement Escrow, Taproot Aggregator, and Ordinal Oracle.
76. Compute `proof_hash = SHA3-256(proof document with proof_hash="")`.
77. Write Settlement Proof to persistent settlement log.
78. Publish Settlement Proof to Ordinal Oracle for registration.
79. Update all commitment records: `commitment_status` → `SETTLED`.
80. Update settlement record: `settlement_status` → `SETTLED`.
81. Emit `settlement.settled` event for Initiator webhook notification.

9.6 Commitment Reversion

Reversion is triggered by: `VALIDATION_FAILED` (if pre-commitment), `COMMITMENT_FAILED` (retries exhausted), `ANCHORING_FAILED` (retries exhausted), `ARBITRATION_RESOLVED_INVALID`, or `EXPIRED` (if committed). Reversion MUST be executed atomically across all rails. The reversion process:

82. For each committed rail: invoke `rail_adapter.revert(commitment_id)`. Reversion unlocks the locked canonical object and returns it to its pre-settlement state. Rail Adapter MUST emit a reversion confirmation.
83. Issue nullifier to ordinal registry: `ordinal_oracle.issue_nullifier(ordinal, canonical_id)`. The nullifier prevents the ordinal from being reused.
84. Update all commitment records: `commitment_status` → `REVERTED`.
85. Update settlement record: `settlement_status` → `REVERTED`.
86. Log `REVERTED` entry with error code and actor.
87. Notify Initiator via webhook or polling response.

10. Settlement Proof Verification

10.1 Verification Algorithm

The following pseudocode is the normative verification algorithm for a Settlement Proof. Any conforming implementation MUST produce the same result for the same inputs.

```
function verify_settlement_proof(proof: SettlementProof) -> VerificationResult: // Step 1: Verify proof
integrity (self-hash check) let expected_hash = SHA3_256(serialize_proof_without_hash_field(proof))
if expected_hash != proof.proof_hash: return
VerificationResult.fail(code="PROOF_HASH_MISMATCH", reason="proof_hash does not match
recomputed hash of proof document") // Step 2: Verify each rail anchor against on-chain data for
rail_id, anchor in proof.rail_anchors: let on_chain_tx = query_rail_transaction(rail_id, anchor.tx_hash)
if on_chain_tx == null: return VerificationResult.fail(code="ANCHOR_NOT_FOUND",
reason=f"tx_hash {anchor.tx_hash} not found on rail {rail_id}") if on_chain_tx.block_height !=
anchor.block_height: return VerificationResult.fail(code="ANCHOR_BLOCK_MISMATCH",
reason=f"block_height mismatch on rail {rail_id}") let current_depth =
get_confirmation_depth(rail_id, anchor.block_height) if current_depth <
get_required_confirmations(rail_id): return
VerificationResult.fail(code="INSUFFICIENT_CONFIRMATIONS", reason=f"rail {rail_id} has
{current_depth} confirmations; requires {required}") if anchor.finality_status != "FINALIZED":
return VerificationResult.fail(code="NOT_FINALIZED", reason=f"rail {rail_id} finality_status is
not FINALIZED") // Step 3: Verify Taproot Root from commitment leaves let sorted_leaves =
lexicographic_sort(proof.commitment_leaves) let recomputed_TR =
compute_merkle_root(sorted_leaves) if recomputed_TR != proof.taproot_root: return
VerificationResult.fail(code="TAPROOT_ROOT_MISMATCH", reason="recomputed taproot_root
from commitment_leaves does not match") // Step 4: Verify Merkle path for each commitment leaf for
i, leaf_hash in enumerate(proof.commitment_leaves): let path_valid = verify_merkle_path(leaf_hash,
proof.merkle_path, proof.taproot_root) if not path_valid: return
VerificationResult.fail(code="MERKLE_PATH_INVALID", reason=f"merkle path verification
failed for leaf {i}") // Step 5: Verify ordinal proof let ordinal_valid =
verify_ordinal_proof(proof.ordinal_proof) if not ordinal_valid: return
VerificationResult.fail(code="ORDINAL_PROOF_INVALID", reason="ordinal proof failed: non-
monotonic or duplicate ordinal") // Step 6: Verify each witness signature for sig in proof.witness_sigs:
let signer_key = resolve_signer_key(sig.signer_id) let sig_valid = Ed25519_verify(signer_key,
proof.proof_hash, sig.signature) if not sig_valid: return
VerificationResult.fail(code="WITNESS_SIG_INVALID", reason=f"witness signature from
{sig.signer_id} is invalid") // Step 7: If S5, verify identity continuity proof if
proof.settlement_category == "S5": let cont_valid =
verify_identity_continuity_proof(proof.identity_continuity_proof) if not cont_valid: return
VerificationResult.fail(code="CONTINUITY_PROOF_INVALID", reason="identity continuity
chain verification failed") // Step 8: All checks passed return VerificationResult.success(
proof_id=proof.proof_id, settlement_id=proof.settlement_id, verified_at=now() )
```

10.2 Verification Failure Codes

Failure Code	Meaning	Verifier Action
PROOF_HASH_MISMATCH	The proof_hash field does not match the SHA3-256 of the proof document. The proof has been tampered with.	Reject proof; report tamper evidence; do not accept settlement.
ANCHOR_NOT_FOUND	The anchor transaction hash was not found on the specified rail. The anchor was never published, or the wrong rail was queried.	Query multiple RPC endpoints; if consistently absent, reject.
ANCHOR_BLOCK_MISMATCH	The anchor transaction exists but is in a different block than claimed. Possible reorg after settlement.	Query current chain state; if reorg: settlement may be invalid.
INSUFFICIENT_CONFIRMATIONS	The anchor block does not yet have the required confirmation depth. Verifier queried too early.	Wait and re-verify after confirmation depth is achieved.
NOT_FINALIZED	Rail's finality_status is not FINALIZED at the time of verification.	Wait for finality; re-verify.
TAPROOT_ROOT_MISMATCH	Recomputed TR from commitment_leaves does not match proof.taproot_root. Commitment leaves may be incomplete or forged.	Reject proof; report anomaly.
MERKLE_PATH_INVALID	Merkle authentication path does not produce the Taproot Root when walked from the leaf.	Reject proof; report anomaly.
ORDINAL_PROOF_INVALID	The ordinal proof fails monotonicity or duplication checks. May indicate a replayed or forged settlement.	Reject proof; check ordinal registry for nullifier.
WITNESS_SIG_INVALID	A witness signature is not a valid Ed25519 signature over the proof_hash by the claimed signer.	Reject proof; report; check if signer key was compromised.
CONTINUITY_PROOF_INVALID	(S5 only) The identity continuity chain is broken or forged. The identity event cannot be trusted.	Reject; escalate to identity authority.

10.3 Light Client Verification

A light client with limited on-chain access (e.g., a mobile wallet or embedded device) can verify a settlement proof using only the Taproot Root published on one rail and the Merkle authentication path, without downloading full block data:

88. Obtain the Taproot Root from any public rail (query the rail's RPC for the anchor tx's OP_RETURN data or contract event log).
89. Receive the Settlement Proof from the URIB API ([GET /v1/settlements/{id}/proof](#)).
90. Re-derive the specific commitment leaf hash from the known commitment fields (which the light client knows from its own transaction history).
91. Walk the merkle_path from the leaf hash, at each step hashing with the sibling in the correct position, to arrive at the root.
92. Assert the recomputed root equals the on-chain Taproot Root obtained in Step 1.
93. Verify proof_hash (Step 1 of Section 10.1).
94. Verify at least one witness_sig (Settlement Escrow or Taproot Aggregator).

This 7-step light verification requires only one on-chain query (for the TR) and $O(\log N)$ Merkle hash operations, making it suitable for resource-constrained clients.

10.4 ZK-Proof Extension (Optional)

For privacy-preserving settlements where intermediate state must not be disclosed, a ZK-SNARK extension is defined as follows (informational; not required for v1 conformance):

- A Groth16 or PLONK ZK-SNARK is produced over the verification circuit defined in Section 10.1 Steps 2–6.
- The public inputs are: `proof_hash`, `taproot_root`, and the set of `rail_ids`.
- The private inputs (witness) are: all `state_hash_from`, `state_hash_to`, commitment content, and finality attestation details.
- The verifier only sees the public inputs and can verify the SNARK proof in $O(1)$ time, confirming that the verification circuit passed without seeing any intermediate state.
- ZK Settlement Proofs are marked with `"proof_type": "ZK-SNARK-GROTH16"` and include the SNARK proof bytes in a `zk_proof` field.

11. Settlement Error Handling & Recovery

11.1 Error Taxonomy

E1xx — Validation Errors

Code	Name	Retriable	Recovery Action	Resulting State
E100	InvalidSignature	No	Initiator must fix signature or canonical_id and resubmit with new settlement_id.	VALIDATION_FAILED
E101	StaleOrdinal	No	Initiator must obtain a fresh ordinal from the Ordinal Oracle and resubmit.	VALIDATION_FAILED
E102	DuplicateSettlement	No	Wait for the active settlement for canonical_id to complete (SETTLED or REVERTED) before resubmitting.	VALIDATION_FAILED
E103	SemanticMismatch	No	Verify SID is correct for the canonical object type; verify target rail supports the type.	VALIDATION_FAILED
E104	UnauthorizedInitiator	No	Verify initiator_id ownership; obtain delegation proof if needed.	VALIDATION_FAILED
E105	RailUnavailable	Yes	Retry after brief wait; contact rail operator if persistent.	VALIDATION_FAILED (or retry VALIDATING)
E106	ExpiredRequest	No	Resubmit with a new expires_at that accounts for all rail finality times.	VALIDATION_FAILED or EXPIRED
E107	ConflictingCommitment	No	Identify and resolve the conflicting active commitment before resubmitting.	VALIDATION_FAILED

E2xx — Commitment Errors

Code	Name	Retriable	Recovery Action	Resulting State
E200	CommitmentWriteFailed	Yes (≤ 3)	Retry with backoff. After 3 failures → REVERTING.	COMMITMENT_FAILED → REVERTING
E201	NullifierConflict	No	Ordinal was already nullified. Obtain new ordinal; resubmit.	COMMITMENT_FAILED → REVERTING
E202	EscrowLockFailed	Yes (≤ 3)	Retry lock operation. If persistent, check escrow contract state.	COMMITMENT_FAILED → REVERTING
E203	TaprootComputationFailed	Yes (≤ 3)	Retry TR computation. If persistent, alert Taproot Aggregator operator.	COMMITMENT_FAILED → REVERTING

E3xx — Anchoring Errors

Code	Name	Retriable	Recovery Action	Resulting State
E300	RailAnchorTimeout	Yes (≤ 3)	Retry anchor submission. Check rail mempool/network status.	ANCHORING_FAILED → REVERTING
E301	TaprootPublicationFailed[rail]	Yes (≤ 3)	Retry publication on the failing rail. If persistent → REVERTING.	ANCHORING_FAILED → REVERTING
E302	RailAdapterUnresponsive	Yes (≤ 3)	Switch to backup RPC endpoint; retry. Alert if Rail Adapter fully down.	ANCHORING_FAILED → REVERTING
E303	InsufficientGas (Ethereum)	Yes (≤ 3)	Resubmit with 1.5x gas limit estimate.	ANCHORING_FAILED → REVERTING

Code	Name	Retriable	Recovery Action	Resulting State
E304	MempoolRejection (Bitcoin)	Yes (≤ 3)	Bump fee via RBF. Resubmit anchor tx.	ANCHORING_FAILED → REVERTING

E4xx — Finality Errors

Code	Name	Retriable	Recovery Action	Resulting State
E400	FinalityTimeout	No	Settlement EXPIRED. Re-anchor if possible; otherwise REVERT and resubmit.	EXPIRED → (REVERTING if committed)
E401	ReorgDetected	Yes (≤ 3)	Re-anchor on affected rail (Section 11.4). Resume PENDING_FINALITY.	ANCHORING (re-anchor) → PENDING_FINALITY
E402	FinalityWatcherFailure	Yes	Restart Finality Watcher; resume from last known confirmed depth.	PENDING_FINALITY (continue)
E403	OrphanedAnchor	Yes (≤ 3)	Anchor tx not in canonical chain. Re-anchor (Section 11.4).	ANCHORING (re-anchor)

E5xx — Dispute Errors

Code	Name	Retriable	Recovery Action	Resulting State
E500	ChallengeBondInsufficient	No (challenger must top up)	Challenger must increase bond and resubmit challenge.	OPTIMISTIC (challenge rejected)
E501	ChallengeWindowExpired	No	Challenge window is closed. Settlement proceeds normally.	OPTIMISTIC (challenge rejected)
E502	ArbitrationTimeout	Escalate	Escalate to admin. Settlement remains in ARBITRATION.	ARBITRATION (escalated)
E503	CounterProofInvalid	No	Challenger must provide valid, structurally correct counter-proof.	OPTIMISTIC (challenge rejected)

E6xx — Settlement Engine Internal Errors

Code	Name	Retriable	Recovery Action	Resulting State
E600	InternalStateCorruption	No	Alert on-call; halt engine; manual recovery required.	REVERTING (emergency)
E601	LogWriteFailed	Yes (≤ 3)	Retry log write. If persistent, halt and alert (audit trail broken).	Current state (paused)
E602	ProofAssemblyFailed	Yes (≤ 3)	Retry proof assembly. All data is present in the commitment store.	FINALIZING (retry)

11.2 Retry Logic

For retrieable errors, the Settlement Engine implements exponential backoff with jitter:

```
function backoff_wait(attempt: int) -> duration_seconds: base = 2 // seconds max_wait = 128 // seconds
raw = base * (2 ** attempt) // 2, 4, 8, 16, 32, 64, 128 jitter = random_uniform(0, base) return
min(raw + jitter, max_wait) // Retry schedule (seconds): ~2, ~4, ~8, ~16 (max 3 retries = 4 total attempts)
// With jitter: ~2-4, ~4-6, ~8-10, ~16-18
```

11.3 Partial Failure Recovery (Multi-Rail S3)

When N rails succeed and M rails fail during S3 anchoring or finality watch:

- **revert_on_partial_failure = true (default):** Settlement Engine reverts ALL rails including those that succeeded. Issues nullifier. Settlement → REVERTED. Initiator must resubmit with a corrected request or after failed rail recovers.
- **revert_on_partial_failure = false:** Failed rails are individually marked as PENDING_RETRY with their specific error code. Succeeded rails remain FINALIZED. The settlement as a whole does NOT reach SETTLED. A retry mechanism re-attempts the failed rails with exponential backoff. If a failed rail subsequently succeeds, the settlement continues toward SETTLED. If it never succeeds (persistent failure), the operator must manually resolve: either accept partial settlement (non-standard, requires governance approval) or fully revert.

11.4 Reorg Recovery

The following procedure applies when a Finality Watcher detects a blockchain reorganization that excludes the anchor transaction:

95. **Detection:** Finality Watcher observes that the anchor block is no longer in the canonical chain (block_hash at anchor_block_height has changed). Emits E401 ReorgDetected.
96. **Re-anchor counter increment:** Settlement Engine increments `re_anchor_attempt[rail_id]`. If `re_anchor_attempt[rail_id] ≥ 3`: emit E401 final, transition to REVERTING.
97. **Re-submission:** Rail Adapter submits a new anchor transaction with the same TR (Taproot Root is deterministic and unchanged). For Bitcoin: use RBF with increased fee. For Ethereum: resubmit to next available block with fresh nonce.
98. **Resume PENDING_FINALITY:** After the new anchor tx is mined, Finality Watcher resumes confirmation depth monitoring from the new anchor block.
99. **Log update:** A SettlementLogEntry with event="reorg_detected" and event="re_anchor_submitted" MUST be written for each reorg event.

11.5 Settlement Resumption

If the Settlement Engine is restarted (crash, maintenance, upgrade) while settlements are in flight, the following resumption procedure applies:

100. On startup, Settlement Engine queries the settlement store for all settlements in non-terminal states: SUBMITTED, VALIDATING, COMMITTING, ANCHORING, PENDING_FINALITY, OPTIMISTIC, CHALLENGED, ARBITRATION, FINALIZING, REVERTING.
101. For each in-flight settlement, the engine determines the current state from the durable state record.
102. **SUBMITTED / VALIDATING:** Re-run validation from the beginning (idempotent; validators re-execute the same checks).
103. **COMMITTING:** Re-run commitment issuance. Idempotency key = settlement_id; if commitments already written, skip and advance to ANCHORING.
104. **ANCHORING:** Re-submit TR to any rails where confirmation has not yet been received. Already-confirmed rails are skipped.
105. **PENDING_FINALITY:** Restart Finality Watchers for all rails that have not yet attested. Watchers resume from the known anchor_block_height.
106. **OPTIMISTIC:** Check if challenge_window_expires_at has passed. If yes: advance to FINALIZING. If no: restart challenge window timer and Finality Watchers.
107. **FINALIZING:** Re-run proof assembly (idempotent; all data is in the commitment store).
108. **REVERTING:** Re-run reversion for any rails not yet reverted. Reversion MUST be completed regardless of engine restart count.

12. Settlement Observability & Auditing

12.1 Settlement Log Schema

Every state transition **MUST** produce a `SettlementLogEntry` (schema defined in Section 3.6). Logs **MUST** be written synchronously before the state transition is committed to the settlement store. A state transition **MUST NOT** be recorded if the log write fails (E601).

Hash Chaining: Each log entry includes `prev_log_hash` = SHA3-256 of the preceding entry. The first entry of any settlement chain uses `prev_log_hash` = SHA3-256("GENESIS"). This produces a tamper-evident chain: modification of any historical entry invalidates all subsequent `prev_log_hash` values, making tampering detectable during audit.

12.2 Metrics to Expose

All URIB Settlement Engine deployments **MUST** expose the following metrics via a standard telemetry endpoint (Prometheus format RECOMMENDED):

Metric Name	Type	Labels	Description
<code>settlement_requests_total</code>	Counter	<code>category</code> , <code>rail_src</code> , <code>rail_dst</code>	Total settlement requests received since engine start.
<code>settlement_success_total</code>	Counter	<code>category</code>	Total settlements that reached SETTLED state.
<code>settlement_failed_total</code>	Counter	<code>error_code</code> , <code>terminal_state</code>	Total settlements that reached a terminal failure state.
<code>settlement_duration_seconds</code>	Histogram	<code>category</code> , <code>rail_src</code> , <code>rail_dst</code>	End-to-end settlement duration from SUBMITTED to SETTLED. Buckets: 1s, 10s, 60s, 300s, 900s, 3600s, 7200s, +Inf.
<code>settlement_in_flight</code>	Gauge	<code>state</code>	Current count of settlements in each non-terminal state.
<code>finality_wait_seconds</code>	Histogram	<code>rail</code>	Duration of PENDING_FINALITY state per rail. Buckets: 1s, 10s, 60s, 300s, 900s, 3600s, 7200s.

Metric Name	Type	Labels	Description
dispute_rate	Gauge	(none)	Rolling 24-hour rate of disputes per 1000 settlements.
commitment_revert_total	Counter	error_code	Total bridge commitments reverted (by error code triggering reversion).
validation_failure_total	Counter	check_id, error_code	Total validation failures by specific check (CANON-1, AUTH-2, etc.).
reorg_detected_total	Counter	rail	Total blockchain reorganizations detected by Finality Watchers.
taproot_computation_duration_seconds	Histogram	leaf_count	Duration of Taproot Root computation by number of leaves.
proof_assembly_duration_seconds	Histogram	category	Duration of Settlement Proof assembly.

12.3 Audit Trail Requirements

- All state transitions **MUST** be immutably logged with: actor identity (`actor_id`), timestamp (ISO 8601 UTC, millisecond precision), triggering event name, and `from_state/to_state`.
- Settlement Proofs **MUST** be retained for a minimum of **7 years** from the `settled_at` timestamp. Long-term storage **MUST** use a content-addressed or cryptographically integrity-protected store.
- Dispute Records **MUST** be retained **indefinitely**. Dispute records are legal evidence and **MUST** never be deleted.
- Log entries **MUST** be hash-chained (see Section 12.1) to provide tamper evidence. The chain integrity **MUST** be verified by automated audit jobs running at minimum daily frequency.
- Log archives **MUST** be replicated to at least 2 geographically distinct storage locations.
- Access to write settlement logs **MUST** be restricted to Settlement Engine components via signed operations. No human operator **MUST** have direct write access to the log store.

12.4 Settlement Dashboard (Informational)

A compliant settlement dashboard SHOULD expose the following key views:

- **Live Settlement Funnel:** Real-time count of settlements in each state (SUBMITTED → VALIDATING → COMMITTING → ANCHORING → PENDING_FINALITY → FINALIZING → SETTLED). Visualized as a funnel or Sankey diagram showing drop-off at each stage.
- **Latency Percentiles:** P50, P95, P99 settlement duration by category and rail combination. Displayed as time series with 1-hour granularity.
- **Error Rate by Code:** Rolling 1-hour error rate per error code (E100–E602). Alert threshold: any error code exceeding 1% of settlement volume.
- **Dispute Rate Trend:** 7-day rolling dispute rate per 1000 settlements. Alert threshold: >0.1% dispute rate.
- **Rail Health Status:** Per-rail health indicator: GREEN (healthy), YELLOW (degraded: elevated latency or error rate), RED (unavailable: Rail Adapter not responding). Derived from Rail Adapter healthcheck and Finality Watcher metrics.
- **Commitment Reversion Rate:** Percentage of settlements that resulted in REVERTED state. Alert threshold: >0.5%.

13. Settlement API Reference

API Version:

All endpoints are prefixed with

/v1/

. The base URL is deployment-specific. Rate limits are per-endpoint per API key. All requests MUST include

Authorization: Bearer {api_key}

and

Content-Type: application/json

headers. All responses include

X-Request-ID

for tracing and

X-URIB-Version: 1.0

13.1 Submit Settlement Request

POST /v1/settlements

Submits a new settlement request. The engine validates the request schema and returns HTTP 202 if accepted. The settlement is NOT yet VALIDATING at this point; validation is asynchronous.

```
// Request Body: SettlementRequest (see Section 3.1 for full schema) { "settlement_id": "3f7a9c2e-1b4d-4e8f-a0c5-d2e6f3a1b7c9", "settlement_category":"S2", ... (all required fields per Section 3.1) } //
HTTP 202 Accepted — Settlement queued { "settlement_id": "3f7a9c2e-1b4d-4e8f-a0c5-d2e6f3a1b7c9", "status": "SUBMITTED", "submitted_at": "2026-07-09T01:58:00.123Z" } //
HTTP 400 Bad Request — Schema invalid or missing required fields { "error_code": "INVALID_REQUEST", "error_message": "Field 'initiator_sig' is required and missing", "field": "initiator_sig" } // HTTP 409 Conflict — Duplicate settlement for canonical_id { "error_code": "E102", "error_message": "An active settlement already exists for canonical_id", "canonical_id": "c-hash:sha3-256:a3f1c9e2b7d4...", "active_settlement_id": "abc12345-..." } // HTTP 422 Unprocessable — Settlement category/option mismatch { "error_code": "INVALID_REQUEST", "error_message": "settlement_category S4 requires optimistic=true" } // Rate Limit: 100 requests/minute per API key.
HTTP 429 on excess.
```

13.2 Get Settlement Status

GET /v1/settlements/{settlement_id}

Returns the current status and details of a settlement. Polling this endpoint is the primary status monitoring mechanism. Implementations SHOULD also offer webhooks (not specified in this version).

```
// HTTP 200 OK { "settlement_id": "3f7a9c2e-1b4d-4e8f-a0c5-d2e6f3a1b7c9", "status":  
"PENDING_FINALITY", "settlement_category": "S2", "canonical_id": "c-hash:sha3-  
256:a3f1c9e2b7d4...", "created_at": "2026-07-09T01:58:00.000Z", "updated_at": "2026-07-  
09T02:00:15.456Z", "current_stage": "Waiting for Bitcoin 6-block confirmation (4/6 so far)",  
"commitments": [ { "commitment_id": "bc:urib:v1:9e3a7f2c...", "commitment_status":  
"ANCHORED", "rail_from": "bitcoin:mainnet", "rail_to": "ethereum:mainnet",  
"taproot_root": "9a2c7e5f..." } ], "proof": null // proof is populated only when status ==  
"SETTLED" } // HTTP 404 Not Found { "error_code": "E600", "error_message": "Settlement not  
found", "settlement_id": "3f7a9c2e-..." } // Rate Limit: 300 requests/minute per API key.
```

13.3 Get Settlement Proof

GET /v1/settlements/{settlement_id}/proof

Returns the full Settlement Proof for a completed settlement. Only available when settlement is in SETTLED or ARCHIVED state.

```
// HTTP 200 OK — Returns full SettlementProof (see Section 3.3) // HTTP 404 Not Found —  
Settlement does not exist // HTTP 409 Conflict — Settlement not yet in SETTLED state { "error_code":  
"SETTLEMENT_NOT_SETTLED", "error_message": "Settlement is not yet in SETTLED state",  
"current_status": "PENDING_FINALITY" } // Rate Limit: 60 requests/minute per API key. Proofs are  
cacheable.
```

13.4 Submit Challenge

POST /v1/settlements/{settlement_id}/challenge

Submits a dispute challenge against an OPTIMISTIC settlement. Challenger MUST have already posted the challenge bond on-chain before calling this endpoint.

```
// Request Body { "challenger_id": "did:urib:l:z6Mkp9r...Abc",  
"challenge_bond_tx": "0xd3c4b5a6e7f8...", "counter_proof": { "proof_type":  
"state_hash_mismatch", "claimed_state_hash": "f2a8c3e7...", "actual_state_hash": "c9e1a7f3...",  
"evidence_tx_hash": "0xb8a7c6e5...", "evidence_rail": "ethereum:mainnet", "challenger_sig":  
"base64url:MEUCIQDc..." } } // HTTP 202 Accepted — Challenge received and queued for validation  
{ "challenge_id": "dr:urib:v1:2c4e6a8f...", "status": "CHALLENGED",  
"arbitration_starts_at": "2026-07-09T03:16:00.000Z" } // HTTP 409 Conflict — Challenge window has  
expired { "error_code": "E501", "error_message": "Challenge window expired at 2026-07-  
09T02:58:00.000Z", "expired_at": "2026-07-09T02:58:00.000Z" } // HTTP 402 Payment Required —  
Insufficient challenge bond { "error_code": "E500", "error_message": "Challenge bond
```

```
insufficient", "required_bond": "0.01 ETH", "submitted_bond": "0.001 ETH" } // Rate Limit: 10 requests/minute per API key.
```

13.5 Verify Settlement Proof

POST /v1/settlements/verify

Accepts a Settlement Proof and runs the full verification algorithm (Section 10.1). Returns the verification result. This endpoint is the canonical proof verification interface for integrators and auditors.

```
// Request Body: Full SettlementProof document // HTTP 200 OK — Proof is valid { "valid": true,
"proof_id": "sp:urib:v1:c7e9a3f1...", "verified_at": "2026-07-09T14:00:00.000Z", "checks_passed":
["PROOF_HASH", "RAIL_ANCHORS", "TAPROOT_ROOT", "Merkle_PATHS",
"ORDINAL_PROOF", "WITNESS_SIGS"] } // HTTP 422 Unprocessable Entity — Proof is invalid {
"valid": false, "failure_code": "TAPROOT_ROOT_MISMATCH", "failure_reason":
"Recomputed taproot_root 'abc...' does not match proof.taproot_root 'def...'", "failed_at_step": 3 } //
Rate Limit: 30 requests/minute per API key.
```

13.6 List Settlements

GET /v1/settlements

Returns a paginated list of settlements matching the specified filters. Supports cursor-based pagination for consistent results on large datasets.

```
// Query Parameters: // canonical_id string — Filter by canonical object C-Hash // status string —
Filter by settlement status (e.g., "SETTLED", "PENDING_FINALITY") // category string — Filter
by category (S1, S2, S3, S4, S5, S6) // rail string — Filter by any rail involvement (source or target)
// from string — ISO 8601 UTC; filter by created_at >= from // to string — ISO 8601 UTC;
filter by created_at <= data-id="2221" to // limit integer — Max results per page (default: 20, max:
100) // cursor string — Pagination cursor from previous response // Example: GET
/v1/settlements?status=SETTLED&category=S2&limit=10 // HTTP 200 OK { "settlements": [ {
"settlement_id": "3f7a9c2e-...", "status": "SETTLED", "settlement_category": "S2",
"canonical_id": "c-hash:sha3-256:a3f1c9e2...", "created_at": "2026-07-09T01:58:00.000Z",
"settled_at": "2026-07-09T03:04:22.789Z" } // ... up to `limit` entries ], "next_cursor":
"eyJsYXN0X2lkIjoim2Y3YTljMmUifQ==", "total": 1847 } // Rate Limit: 60 requests/minute per
API key.
```

14. Settlement Security Considerations

14.1 Front-Running Protection

Settlement requests for sensitive operations (e.g., high-value asset transfers) **MUST** use a commit-reveal pattern to prevent front-running by mempool observers:

109. **Commit phase:** Initiator first submits a *commitment hash* = SHA3-256(settlement_id || payload_hash || nonce) to the Settlement Engine. The nonce is a secret known only to the Initiator.
110. **Reveal phase:** After the commitment hash is acknowledged, the Initiator submits the full Settlement Request (with the nonce) within a short time window (e.g., 30 seconds).
111. The Settlement Engine verifies that the revealed payload matches the commitment hash before proceeding to VALIDATING.

The commit-reveal pattern ensures that even if a malicious actor observes the reveal transaction, they cannot front-run it because the settlement slot was already committed.

14.2 Double-Spend / Double-Settlement Prevention

URIB employs three independent layers of double-settlement prevention:

- **Non-Duplication Invariant (COMMIT-1 check, Section 8.5):** Checked at VALIDATING before any commitment is issued. Prevents two settlements from starting for the same canonical_id simultaneously.
- **Nullifier Registry:** After any REVERTED settlement, a nullifier is issued to the ordinal registry for the canonical_id + ordinal pair. Any future settlement using the same ordinal will fail at CANON-3 (E101). Nullifiers are permanent and never expire.
- **Atomic Escrow Lock:** The Settlement Escrow acquires an exclusive lock on the canonical_id as part of the COMMITTING phase (escrow.lock() is atomic). If two concurrent settlements somehow pass VALIDATING simultaneously (race condition), only the first to acquire the escrow lock succeeds; the second fails with E202 EscrowLockFailed.

14.3 Replay Attack Prevention

- **settlement_id UUID v4:** Every settlement request has a universally unique settlement_id. The Settlement Engine maintains a global settlement_id registry and rejects any duplicate settlement_id with E102. Uniqueness is enforced with a unique constraint on the settlement store's primary key.
- **expires_at enforcement:** Stale settlement requests are rejected at submission if `now() ≥ expires_at - 60s`. This ensures old requests that were delayed in transit cannot be replayed.
- **Ordinal freshness (ORD-1, ORD-2):** Ordinals have an epoch field. Stale epochs (older than max_epoch_lag) are rejected. This provides a time-bounded replay prevention window even if settlement_id uniqueness is somehow circumvented.
- **HTTPS Rail nonce:** Off-chain API deliveries include a UUID nonce with a 24-hour deduplication window at the recipient, preventing replay of captured settlement payloads.

14.4 Commitment Forgery

- All bridge commitments are signed by the Settlement Escrow key at issuance time. A forged commitment without the Escrow's signature will fail at proof verification (WITNESS_SIG_INVALID).
- The Taproot Root is publicly verifiable from on-chain data and the commitment leaves. Forging a TR requires breaking SHA-3-256 (preimage resistance: 2^{256} operations) or finding a Merkle collision (2^{128} operations for second-preimage): computationally infeasible with current technology.
- Multi-validator mode: deployments with N validators require $2/3+1$ validator signatures on the validation attestation. A single compromised validator cannot forge a commitment alone.

14.5 Finality Oracle Manipulation

- Finality Watchers MUST connect to **at least 2 independent RPC endpoints** per rail. This prevents a single compromised or manipulated RPC provider from causing false finality attestations.
- **Byzantine fault tolerance:** If Finality Watcher endpoints disagree on finality (e.g., one reports 6 confirmations, another reports 3), the Settlement Engine MUST wait for all endpoints to reach the required confirmation depth before issuing a Finality Attestation. Disagreement beyond a configurable threshold (default: N/3 endpoints disagree) MUST trigger an alert and escalation.
- All Finality Attestations are signed by the Finality Watcher key. The Finality Watcher key MUST be stored in an HSM and rotated annually.

14.6 Arbitrator Capture

- Dispute Arbitrators MUST implement deterministic resolution logic. Given the same inputs (original commitment + counter_proof), the same output MUST always be produced. Non-deterministic or discretionary arbitration is not permitted.
- **On-chain arbitration preference:** Where possible (Ethereum, XRPL), arbitration logic SHOULD be implemented as a smart contract or on-chain program. This makes the resolution logic auditable, immutable, and non-capture-able by a single operator.
- **Multi-arbitrator quorum:** For settlements above a configurable value threshold (default: \$100,000 equivalent), a quorum of N arbitrators (minimum 3) MUST agree on the resolution. A simple majority (2/3) is required. This prevents capture of a single arbitrator from determining high-value outcomes.
- Arbitration resolution logic version MUST be pinned and its version hash recorded in the Dispute Record. Any upgrade to arbitration logic MUST follow a governance process with public notice period.

15. Conformance & Testing

15.1 Settlement Conformance Checklist

A URIB Settlement Engine implementation is considered conformant when it satisfies all of the following requirements. Items marked [REQUIRED] are mandatory for v1.0 conformance. Items marked [RECOMMENDED] are optional but strongly advised.

#	Requirement	Level	Reference
1	All settlement states (DRAFT through ARCHIVED) are implemented with correct entry/exit conditions and timeout behaviors.	[REQUIRED]	Section 4.1
2	All state transitions from Section 4.2 are implemented, including all error and recovery paths.	[REQUIRED]	Section 4.2
3	All transition guards from Section 4.4 are enforced atomically.	[REQUIRED]	Section 4.4
4	All validation checks (CANON-1 through FIN-3) from Section 8 are executed at VALIDATING phase.	[REQUIRED]	Section 8
5	All error codes (E100–E602) are returned correctly with appropriate HTTP status codes and error messages.	[REQUIRED]	Section 11.1
6	SettlementRequest, BridgeCommitment, SettlementProof, FinalityAttestation, DisputeRecord, and SettlementLogEntry schemas from Section 3 are fully implemented with all field validations.	[REQUIRED]	Section 3
7	All API endpoints from Section 13 (POST /settlements, GET /settlements/{id}, GET /settlements/{id}/proof, POST /settlements/{id}/challenge, POST /settlements/verify, GET /settlements) are implemented with correct request/response schemas and HTTP status codes.	[REQUIRED]	Section 13
8	All metrics from Section 12.2 are exposed in Prometheus-compatible format.	[REQUIRED]	Section 12.2
9	Audit log is hash-chained (prev_log_hash) and tamper-evident. Log integrity is verified by automated daily audit jobs.	[REQUIRED]	Section 12.3
10	Settlement categories S1, S2, S3, S4, and S5 are fully supported. S6 (Disputed Settlement) is mandatory only when S4 is supported.	[REQUIRED]	Section 5
11	Retry logic with exponential backoff (base 2s, max 128s, max 3 retries) is implemented for all retrievable error stages.	[REQUIRED]	Section 7.3

#	Requirement	Level	Reference
12	settlement_id is enforced as a global idempotency key. Duplicate settlement_ids are rejected with E102.	[REQUIRED]	Section 14.3
13	Nullifier registry prevents double-settlement. Nullifiers are permanent and never expire.	[REQUIRED]	Section 14.2
14	Finality Watcher uses ≥ 2 independent RPC endpoints per rail.	[REQUIRED]	Section 14.5
15	Settlement Proof verification algorithm (Section 10.1) is implemented and exposed via POST /settlements/verify.	[REQUIRED]	Section 10.1
16	Settlement resumption after engine restart is supported for all non-terminal states (Section 11.5).	[REQUIRED]	Section 11.5
17	Reorg detection and re-anchor procedure (Section 11.4) is implemented for all blockchain rails.	[REQUIRED]	Section 11.4
18	Settlement Proofs are retained for minimum 7 years; Dispute Records retained indefinitely.	[REQUIRED]	Section 12.3
19	ZK-SNARK settlement proof extension (Section 10.4) is supported.	[RECOMMENDED]	Section 10.4
20	Commit-reveal front-running protection (Section 14.1) is implemented for high-value settlements.	[RECOMMENDED]	Section 14.1
21	Multi-validator mode with 2/3+1 quorum is supported.	[RECOMMENDED]	Section 14.4
22	Multi-arbitrator quorum (≥ 3 arbitrators) is supported for high-value dispute resolution.	[RECOMMENDED]	Section 14.6

15.2 Test Vectors

C-Hash Computation Reference Vector

```
// Canonical object fields (in deterministic serialization order): semantic_id =
"sid:urib:v1:token:fungible:erc20" rail_id = "ethereum:mainnet" owner =
"0xCafeBabe1234567890ABCDEFcafebabe12345678" amount = "10000000000000000000"
token_address = "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48" epoch = 194 sequence
= 10041 // Serialization (canonical JSON, keys sorted alphabetically): canonical_json =
'{"amount":"10000000000000000000","epoch":194,"owner":"0xCafeBabe...","rail_id":"ethereum:mainnet",
,"semantic_id":"sid:urib:v1:token:fungible:erc20","sequence":10041,"token_address":"0xA0b86..."}' //
C-Hash: c_hash = "c-hash:sha3-256:" + hex(SHA3_256(canonical_json)) c_hash = "c-hash:sha3-
256:9f3a1c7e2b4d5f8a3c6e9f1b2d4a7e3c..."
```

Taproot Root Computation Reference Vector (4 Leaves)

```
// Input leaves (raw SHA3-256 hashes of commitment content): L1 =
"a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2" L2 =
"b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3" L3 =
"c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4" L4 =
"d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5" // Step 1: Sort
lexicographically (already sorted in this example) sorted = [L1, L2, L3, L4] // Step 2: Pairwise hash
(left,right sorted lexicographically before hashing) H12 = SHA3_256(sort(L1, L2)[0] || sort(L1, L2)[1])
= SHA3_256(L1 || L2) // L1 < L2, so L1 is left =
"e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6" H34 = SHA3_256(L3 || L4)
// L3 < L4 = "f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7" // Step 3:
Root TR = SHA3_256(H12 || H34) // H12 < H34 =
"9a2c7e5f3b1d8a4c6e2f9b3d1a7c5e4f8b2d6a9c3e7f1b5d8a4c6e2f9b3d1a7c" // Merkle path for L1: path
= [ { sibling: L2, position: "right" }, // L1 + L2 = H12 { sibling: H34, position: "right" } // H12 +
H34 = TR ]
```

Settlement Proof Hash Computation Reference Vector

```
// Given a SettlementProof document, set proof_hash = "" and serialize: proof_without_hash =
'{"canonical_id":"...","commitment_leaves":[...],...,"proof_hash":"","...}' // Compute: proof_hash =
hex(SHA3_256(proof_without_hash)) =
"7f3a1c9e2b5d8f4a3c6e1f9b2d5a8c4e7f3b1d9a2c5e8f4b7d3a1c9e2b5d8f4a" // Verify: set
proof.proof_hash = above value and re-serialize (should be stable).
```

Merkle Path Verification Reference Vector

```
// Given: leaf_hash = L1, merkle_path = [{L2, right}, {H34, right}], TR = "9a2c..." function
verify_merkle_path(leaf, path, root): current = leaf // "a1b2c3d4..." for step in path: if step.position
== "right": current = SHA3_256(current || step.sibling) else: // "left" current =
SHA3_256(step.sibling || current) return current == root // Trace: // current = L1 = "a1b2..." // Step 1
(sibling=L2, pos=right): current = SHA3_256(L1 || L2) = H12 = "e5f6..." // Step 2 (sibling=H34,
pos=right): current = SHA3_256(H12 || H34) = TR = "9a2c..." // assert "9a2c..." == TR → TRUE ✓
```

15.3 Integration Test Scenarios

All conformant implementations MUST pass the following integration test scenarios. Test scenarios are defined at the behavioral level; specific test data is provided in the URIB Test Suite repository.

#	Scenario	Expected Outcome
IT-01	Happy path S1: Ethereum intra-rail settlement	→ SETTLED within 20 min; valid proof produced
IT-02	Happy path S2: Bitcoin → Ethereum cross-rail	→ SETTLED within 75 min; dual rail anchors in proof
IT-03	Happy path S3: BTC + ETH + XRPL fan-out	→ SETTLED; 3-leaf Taproot tree; all 3 rail_anchors in proof
IT-04	Happy path S4: Optimistic Solana → ETH, no challenge	→ OPTIMISTIC → SETTLED after challenge_window expiry
IT-05	Happy path S5: Key rotation, 3 active rails	→ SETTLED; identity_continuity_proof present in settlement proof
IT-06	S4 optimistic with valid challenge → REVERTED	→ CHALLENGED → ARBITRATION → ARBITRATION_RESOLVED_INVALID → REVERTED; challenger bond returned
IT-07	S4 optimistic with invalid challenge (insufficient bond)	Challenge rejected (E500); settlement resumes → SETTLED
IT-08	S4 optimistic with structurally invalid counter_proof	Challenge rejected (E503); settlement resumes → SETTLED; challenger bond forfeited
IT-09	Validation failure: invalid initiator_sig (E100)	→ VALIDATION_FAILED immediately; error_code=E100
IT-10	Validation failure: stale ordinal (E101)	→ VALIDATION_FAILED; error_code=E101
IT-11	Validation failure: duplicate settlement for canonical_id (E102)	→ VALIDATION_FAILED; error_code=E102; active_settlement_id returned
IT-12	Validation failure: semantic mismatch — NFT type to fungible-only rail (E103)	→ VALIDATION_FAILED; error_code=E103
IT-13	Validation failure: unauthorized initiator (E104)	→ VALIDATION_FAILED; error_code=E104
IT-14	Validation failure: rail unavailable (E105)	→ VALIDATION_FAILED; error_code=E105 (retriable)
IT-15	Commitment write failure with retry: DB unavailable for 2 attempts, then recovers	→ COMMITMENT_FAILED × 2 → COMMITTING (retry) → ANCHORING → SETTLED

#	Scenario	Expected Outcome
IT-16	Commitment write failure: 3 consecutive DB failures	→ COMMITMENT_FAILED × 3 → REVERTING → REVERTED
IT-17	Anchoring timeout on Bitcoin rail (E300): simulated RPC timeout	→ ANCHORING_FAILED; retry attempts logged; after 3 failures → REVERTING → REVERTED
IT-18	Finality timeout (E400): Bitcoin finality_timeout exceeded	→ EXPIRED; if committed → REVERTING → REVERTED
IT-19	S3 partial failure with revert_on_partial_failure=true: 1 of 3 rails fails finality	All 3 rails reverted → REVERTED; all commitment_status → REVERTED
IT-20	S3 partial failure with revert_on_partial_failure=false: 1 of 3 rails fails finality	Failed rail marked PENDING_RETRY; successful rails remain FINALIZED; settlement NOT SETTLED
IT-21	Engine restart mid-PENDING_FINALITY: kill engine; restart; verify settlement resumes	Finality Watchers restart from last known confirmation depth; settlement completes → SETTLED
IT-22	Reorg detection and re-anchor: simulate Bitcoin orphan block at anchor height	E401 detected; re-anchor submitted; confirmation count restarted; → SETTLED after 6 new confirmations
IT-23	S5 identity continuity: key rotation on 3 rails; verify continuity chain in proof	→ SETTLED; identity_continuity_proof contains full event chain from genesis; all 3 rail_anchors present
IT-24	Proof verification: submit valid SettlementProof to POST /settlements/verify	HTTP 200; valid=true; all 6 check stages passed
IT-25	Proof verification: submit tampered proof (modified state_hash_to)	HTTP 422; valid=false; failure_code=TAPROOT_ROOT_MISMATCH or PROOF_HASH_MISMATCH
IT-26	Settlement_id idempotency: submit same settlement_id twice	Second submission returns HTTP 409; error_code=E102
IT-27	Nullifier registry: attempt settlement of a REVERTED canonical_id with same ordinal	→ VALIDATION_FAILED; error_code=E101 (ordinal nullified)
IT-28	Light client verification: verify using only Taproot Root + merkle_path	Verification passes without full block data access

Appendix A — Settlement State Machine Formal Specification (Pseudocode)

The following pseudocode is the normative implementation of the settlement state machine transition function. All conformant implementations MUST produce equivalent behavior. The function is pure: given a Settlement record and an Event, it returns a new Settlement record with updated state and a list of side effects to execute.

```
// Settlement State Machine — Normative Transition Function // Returns: (new_settlement: Settlement,
effects: list[Effect]) function transition(settlement: Settlement, event: Event) -> (Settlement, list[Effect]):
match (settlement.status, event):    // — DRAFT

—— (DRAFT, Submit(request)):    guard: is_valid_uuid_v4(request.settlement_id)    AND
schema_valid(request)    AND now() < request.expires_at    AND NOT
exists_in_registry(request.settlement_id)    new_status = SUBMITTED    effects = [
WriteSettlementRecord(request),    WriteLogEntry(from=DRAFT, to=SUBMITTED,
event="submit"),    EmitHTTP202(settlement_id=request.settlement_id)    ]    // — SUBMITTED

(SUBMITTED, ValidationStart()):    guard: no_active_settlement_exists(settlement.canonical_id)
AND now() < settlement.expires_at    new_status = VALIDATING    effects = [
InvokeValidators(settlement),    WriteLogEntry(from=SUBMITTED, to=VALIDATING,
event="validation_start"),    SetTimeout(validation_timeout, 30)    ]    (SUBMITTED,
DuplicateDetected()):    new_status = VALIDATION_FAILED    effects = [
WriteLogEntry(from=SUBMITTED, to=VALIDATION_FAILED, event="duplicate_detected",
error_code=E102),    NotifyInitiator(error_code=E102)    ]    // — VALIDATING

(VALIDATING, ValidateOK(attestation)):    guard: all_validation_checks_pass(settlement) == TRUE
new_status = COMMITTING    effects = [    StoreValidationAttestation(attestation),
WriteLogEntry(from=VALIDATING, to=COMMITTING, event="validate_ok"),
InvokeSettlementEscrow(settlement, action=LOCK_AND_COMMIT),
SetTimeout(commitment_timeout, 60)    ]    (VALIDATING, ValidateFail(error_code)):
new_status = VALIDATION_FAILED    effects = [
ReleaseOrdinalReservation(settlement.ordinal),    WriteLogEntry(from=VALIDATING,
to=VALIDATION_FAILED,    event="validate_fail", error_code=error_code),
NotifyInitiator(error_code=error_code)    ]    (VALIDATING, Timeout()):    // Identical to
ValidateFail but with timeout error code    return transition(settlement, ValidateFail(error_code=E106))
// — COMMITTING

(COMMITTING, CommitmentIssued(commitment_records)):    guard: count(commitment_records) ==
expected_count(settlement)    AND all_records_written(commitment_records)    AND
canonical_object_locked(settlement.canonical_id)    new_status = ANCHORING    effects = [
SetAllCommitmentStatus(commitment_records, PENDING),    WriteLogEntry(from=COMMITTING,
to=ANCHORING, event="commitment_issued"),    InvokeTaprootAggregator(settlement,
commitment_records),    SetTimeout(anchoring_timeout, 120)    ]    (COMMITTING,
CommitmentWriteFailed(error_code)):    new_status = COMMITMENT_FAILED
settlement.retry_count += 1    effects = [    WriteLogEntry(from=COMMITTING,
to=COMMITMENT_FAILED,    event="commitment_write_failed", error_code=error_code,
metadata={retry_attempt: settlement.retry_count}),    IF settlement.retry_count < 3:
ScheduleRetry(backoff_wait(settlement.retry_count),
action=RetryCommitment(settlement))    ELSE:    transition(settlement, MaxRetriesExceeded())
]    (COMMITMENT_FAILED, RetryCommitment()):    guard: settlement.retry_count < 3
```



```

new_status = COMMITTING    effects = [    WriteLogEntry(from=COMMITMENT_FAILED,
to=COMMITTING,
event="retry_commitment",
metadata={attempt:
settlement.retry_count}),    InvokeSettlementEscrow(settlement, action=LOCK_AND_COMMIT)
]    (COMMITMENT_FAILED, MaxRetriesExceeded()):    new_status = REVERTING    effects = [
WriteLogEntry(from=COMMITMENT_FAILED, to=REVERTING,
event="max_retries_exceeded"),    InvokeReversion(settlement)    ]    // — ANCHORING

```

```

(ANCHORING, TaprootPublished(taproot_root, per_rail_tx_hashes)):    guard:
taproot_root_valid(taproot_root, settlement.commitment_records)    AND
all_rails_acknowledged(per_rail_tx_hashes, settlement.target_rails)    new_status =
PENDING_FINALITY    effects = [    SetAllCommitmentStatus(settlement.commitments,
ANCHORED),    StoreTaprootRoot(settlement, taproot_root),
WriteLogEntry(from=ANCHORING, to=PENDING_FINALITY, event="taproot_published"),    FOR
EACH rail IN settlement.target_rails:    StartFinalityWatcher(settlement.settlement_id, rail,
tx_hash=per_rail_tx_hashes[rail])    SetTimeout(finality_timeout[rail], per_rail_default)    ]
(ANCHORING, AnchorFailed(rail_id, error_code)):    new_status = ANCHORING_FAILED
settlement.anchor_retry_count[rail_id] += 1    effects = [    WriteLogEntry(from=ANCHORING,
to=ANCHORING_FAILED,
event="anchor_failed", error_code=error_code,
metadata={rail: rail_id}),    IF settlement.anchor_retry_count[rail_id] < 3:
ScheduleRetry(backoff_wait(settlement.anchor_retry_count[rail_id]),
action=RetryAnchor(settlement, rail_id))    ELSE:    transition(settlement,
MaxAnchorRetriesExceeded())    ]    (ANCHORING_FAILED, MaxAnchorRetriesExceeded()):
new_status = REVERTING    effects = [WriteLogEntry(...), InvokeReversion(settlement)]    // —
PENDING_FINALITY

```

```

(PENDING_FINALITY, AllFinalitiesConfirmed(attestations)):    guard: count(attestations) ==
count(settlement.target_rails)    AND all_attestations_valid(attestations)    new_status =
FINALIZING    effects = [    SetAllCommitmentStatus(settlement.commitments, FINALIZED),
StoreAttestations(attestations),    WriteLogEntry(from=PENDING_FINALITY, to=FINALIZING,
event="all_finalities_confirmed"),    InvokeProofAssembly(settlement)    ]
(PENDING_FINALITY, OptimisticFirstRail(rail_id, attestation)):    guard:
settlement.settlement_options.optimistic == TRUE    AND finality_attested(rail_id) == TRUE
new_status = OPTIMISTIC    effects = [    SetCommitmentStatus(rail_id, FINALIZED),
SetChallengeWindowExpiry(settlement,    now() +
settlement.settlement_options.challenge_window_seconds),
WriteLogEntry(from=PENDING_FINALITY, to=OPTIMISTIC,
event="optimistic_first_rail"),    StartChallengeWindowTimer(settlement)    ]
(PENDING_FINALITY, FinalityTimeout(rail_id)):    new_status = EXPIRED    effects = [
WriteLogEntry(from=PENDING_FINALITY, to=EXPIRED,
event="finality_timeout",
error_code=E400),    IF has_committed_state(settlement):    transition(settlement, Revert())
ELSE:    NotifyInitiator(error_code=E400)    ]    // — OPTIMISTIC

```

```

(OPTIMISTIC, ChallengeWindowExpired()):    guard: now() >=
settlement.challenge_window_expires_at    AND NOT exists_pending_challenge(settlement)
new_status = FINALIZING    effects = [    SetAllCommitmentStatus(settlement.commitments,
FINALIZED),    WriteLogEntry(from=OPTIMISTIC, to=FINALIZING,
event="challenge_window_expired"),    InvokeProofAssembly(settlement)    ]    (OPTIMISTIC,
ChallengeReceived(challenge)):    guard: now() < settlement.challenge_window_expires_at
AND challenge_bond_confirmed(challenge.bond_tx)    AND
counter_proof_structurally_valid(challenge.counter_proof)    new_status = CHALLENGED    effects
= [    CreateDisputeRecord(settlement, challenge),    LockChallengeBond(challenge.bond_tx),
WriteLogEntry(from=OPTIMISTIC, to=CHALLENGED, event="challenge_received")    ]    // —
CHALLENGED

```

```
(CHALLENGED, ChallengeValidated(dispute_record)):    new_status = ARBITRATION    effects = [
    SetArbitrationExpiry(settlement, now() + arbitration_timeout),
    InvokeArbitrator(dispute_record),    WriteLogEntry(from=CHALLENGED, to=ARBITRATION,
event="challenge_validated")    ]    (CHALLENGED, ChallengeInvalid(error_code)):    new_status =
OPTIMISTIC // challenge rejected; window resumes    effects = [
ReturnChallengeBond(settlement.dispute.challenger_id),    WriteLogEntry(from=CHALLENGED,
to=OPTIMISTIC,
event="challenge_invalid", error_code=error_code)    ]    // —
ARBITRATION
```

```
(ARBITRATION, ResolutionValid(resolution)):    guard: arbitrator_sig_valid(resolution)
new_status = ARBITRATION_RESOLVED_VALID    effects = [
SlashChallengeBond(settlement.dispute.challenger_id),    UpdateDisputeRecord(resolution),
WriteLogEntry(from=ARBITRATION, to=ARBITRATION_RESOLVED_VALID,
event="resolution_valid")    ]    (ARBITRATION, ResolutionInvalid(resolution)):    guard:
arbitrator_sig_valid(resolution)    new_status = ARBITRATION_RESOLVED_INVALID    effects = [
    PenalizeInitiator(settlement.initiator_id),    UpdateDisputeRecord(resolution),
    WriteLogEntry(from=ARBITRATION, to=ARBITRATION_RESOLVED_INVALID,
event="resolution_invalid")    ]    // — ARBITRATION_RESOLVED_*
```

```
(ARBITRATION_RESOLVED_VALID, Resume()):    new_status = PENDING_FINALITY
effects = [    WriteLogEntry(from=ARBITRATION_RESOLVED_VALID,
to=PENDING_FINALITY,
event="resume"),    FOR EACH rail IN
settlement.target_rails WHERE NOT finalized:    StartFinalityWatcher(settlement, rail)    ]
(ARBITRATION_RESOLVED_INVALID, Revert()):    new_status = REVERTING    effects = [
WriteLogEntry(from=ARBITRATION_RESOLVED_INVALID, to=REVERTING,
event="revert"),    InvokeReversion(settlement)    ]    // — FINALIZING
```

```
(FINALIZING, ProofAssembled(proof)):    guard: proof_hash_valid(proof)    AND
count(proof.witness_sigs) >= required_witness_count()    AND all_rail_anchors_verified(proof)
new_status = SETTLED    effects = [    SetAllCommitmentStatus(settlement.commitments,
SETTLED),    PersistSettlementProof(proof),    PublishProofToOrdinalOracle(proof),
WriteLogEntry(from=FINALIZING, to=SETTLED, event="proof_assembled"),
NotifyInitiator(proof=proof),    EmitMetric("settlement_success_total")    ]    (FINALIZING,
ProofAssemblyFailed(error_code)):    // Retry in place; do not change state    effects = [
WriteLogEntry(event="proof_assembly_failed", error_code=E602),    IF retry_count < 3:
ScheduleRetry(action=InvokeProofAssembly(settlement))    ELSE: AlertOnCall("Proof assembly
failed after 3 retries")    ]    // — REVERTING
```

```
(REVERTING, ReversionComplete(nullifier)):    guard: all_rails_reverted(settlement)    AND
nullifier_recorded(nullifier)    AND canonical_object_unlocked(settlement.canonical_id)
new_status = REVERTED    effects = [    SetAllCommitmentStatus(settlement.commitments,
REVERTED),    RecordNullifier(nullifier),    WriteLogEntry(from=REVERTING,
to=REVERTED, event="reversion_complete"),    NotifyInitiator(terminal_state=REVERTED,
error_code=settlement.terminal_error_code),    EmitMetric("commitment_revert_total")    ]    // —
SETTLED
```

```
— (SETTLED, Archive()):    guard: now() >= settled_at + active_retention_period    new_status =
ARCHIVED    effects = [    MoveToLongTermStorage(settlement),
WriteLogEntry(from=SETTLED, to=ARCHIVED, event="archive")    ]    // — GLOBAL TIMEOUT
GUARD
```

```
(any_non_terminal_state, ExpiresAtExceeded()):    guard: now() > settlement.expires_at    effects = [
WriteLogEntry(event="expires_at_exceeded", error_code=E106),    IF
```

```

has_committed_state(settlement):      transition(settlement.with_status(REVERTING), Revert())
ELSE:      transition(settlement, ValidateFail(E106)) // → VALIDATION_FAILED or EXPIRED
] // — DEFAULT

```

```

— (, ): // Illegal transition: log and alert; do not change state      effects = [
WriteLogEntry(event="illegal_transition_attempted",      from=settlement.status,
event_type=event.type),      AlertOnCall("Illegal state machine transition attempted")      ]      return
(settlement, effects) // state unchanged      return (settlement.with_status(new_status), effects)

```

Appendix B — Worked Example: Full End-to-End S2 Settlement Trace

This appendix presents a complete, realistic trace of an S2 (Cross-Rail Bilateral) settlement from DRAFT through to SETTLED, showing every state transition, every data structure at each stage, every relevant timestamp, and the final Settlement Proof. Timestamps are shown with elapsed time from submission.

```

===== URIB S2 SETTLEMENT — COMPLETE END-TO-END TRACE Scenario: 0.5
BTC UTXO bridged to 0.499 WBTC on Ethereum (0.001 BTC deducted as WBTC bridge fee —
intentionally correct this time)
=====

T+0:00.000 STATE: DRAFT → SUBMITTED

SettlementRequest submitted: { "settlement_id":      "b7e3c1a9-4f2d-4e8a-b0c6-d3f7a2e9c1b4",
"settlement_category": "S2", "canonical_id":      "c-hash:sha3-
256:e7a3c1f9b2d4e6a8c3f7b1d9e2a4c6f8...", "semantic_id":      "sid:urib:v1:token:fungible:bitcoin",
"ordinal":      { "rail_id":"bitcoin:mainnet","epoch":842,"sequence":50017 }, "source_rail":
"bitcoin:mainnet", "target_rails":      ["ethereum:mainnet"], "initiator_id":
"did:urib:1:z6Mkf5r9xQvc", "initiator_sig":
"base64url:MEUCIQDh3a7c9f1b2e4d6a8c0f3e7b1d9a2c4f6...", "payload_hash":
"f4c1e9a3b7d2f6e8a0c3f7b1d9e2a4c6f8b0d3e7...", "settlement_options": { "optimistic":false,
"revert_on_partial_failure":true }, "created_at":      "2026-07-09T01:58:00.000Z", "expires_at":
"2026-07-09T05:58:00.000Z" } Engine Response: HTTP 202 Log Entry #0001: { from:DRAFT,
to:SUBMITTED, event:"submit", ts:"01:58:00.123Z" }

===== T+0:00.150
STATE: SUBMITTED → VALIDATING

===== Log Entry #0002:
{ from:SUBMITTED, to:VALIDATING, event:"validation_start", ts:"01:58:00.273Z" } Validation
checks executing: [CANON-1] C-Hash recompute → PASS [CANON-2] payload_hash match →
PASS [CANON-3] ordinal not in registry → PASS (epoch:842, seq:50017 is fresh) [CANON-4]
ordinal sequence > last known (50016) → PASS [SEM-1] SID "sid:urib:v1:token:fungible:bitcoin"
registered → PASS [SEM-2] Both rails accept fungible token type → PASS [SEM-3] No semantic
tag conflicts → PASS [AUTH-1] Initiator key resolved: Ed25519 pub "z6Mkf5r9xQvc..." → PASS
[AUTH-2] initiator_sig verified → PASS [AUTH-3] Initiator owns UTXO a1b2c3d4:0 → PASS
[RAIL-1] bitcoin:mainnet registered → PASS [RAIL-1] ethereum:mainnet registered → PASS
[RAIL-2] BTC Rail Adapter ping: 200 OK (48ms) → PASS [RAIL-2] ETH Rail Adapter ping: 200
OK (31ms) → PASS [RAIL-3] Both adapters accept fungible token → PASS [COMMIT-1] No active
commitment for canonical_id → PASS [COMMIT-2] No active settlement for canonical_id → PASS
[COMMIT-3] Ordinal not nullified → PASS [ORD-1] Epoch 842 >= current_epoch(842) -
max_lag(10) → PASS [ORD-2] seq 50017 > last known 50016 → PASS [ORD-3] Ordinal Oracle

```

sig verified → PASS [FIN-1] expires_at 05:58Z >> expected settle time (~03:10Z) → PASS [FIN-3]
expires_at 05:58Z > now+60s → PASS All 22 checks PASSED. Validation attestation signed by
Validator-01.

T+0:00.850 STATE: VALIDATING → COMMITTING

Log Entry #0003:
{ from:VALIDATING, to:COMMITTING, event:"validate_ok", ts:"01:58:00.973Z" } Settlement
Escrow locks BTC UTXO (outpoint a1b2c3d4:0, value 50,000,000 sats): lock_tx:
"d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6" lock_method:
OP_RETURN commitment inscription Bridge commitments issued atomically: Commitment Leaf 1:
commitment_id: "bc:urib:v1:9c3f7a1e2b4d6f8a0c3e7b1d9f2a4c6..." rail from: "bitcoin:mainnet"
rail to: "ethereum:mainnet" state_hash from: "7c4e1a9f2b8d3e5ca0f3b7d1e9c2a4f6..." // BTC
UTXO locked state_hash to: "f2a8c3e7d1b9f4a60c3e7b1d9f2a4c6e..." // 0.499 WBTC to 0xCafe...
ordinal_from: { rail_id:"bitcoin:mainnet", epoch:842, sequence:50017 } ordinal_to: {
rail_id:"ethereum:mainnet", epoch:194, sequence:10042 } commitment_status: PENDING
taproot_leaf_hash: "e5d3f1a7c2b9e4f60c3e7b1d9f2a4c6..." Log Entry #0004: { from:COMMITTING,
to:ANCHORING, event:"commitment_issued", ts:"01:58:01.234Z" }

T+0:01.250

STATE: COMMITTING → ANCHORING

Taproot
Aggregator receives 1 leaf (S2 single commitment): leaf_hash: "e5d3f1a7c2b9e4f60c3e7b1d9f2a4c6..."
sorted_leaves: ["e5d3f1a7c2b9e4f60c3e7b1d9f2a4c6..."] TR = SHA3_256(leaf_hash) // trivial single-
leaf case: TR == leaf_hash = "e5d3f1a7c2b9e4f60c3e7b1d9f2a4c6..." NOTE: For consistency and
future extensibility, the Taproot Aggregator always applies the full Merkle construction. A single-
leaf tree has TR = SHA3_256(leaf || leaf) per reference implementation. TR =
SHA3_256("e5d3f1..." || "e5d3f1...") = "9a2c7e5f3b1d8a4c6e2f9b3d1a7c5e4f..." BTC Rail Adapter
publishes TR to Bitcoin: method: OP_RETURN in tx btc_anchor_tx:
"a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2" mined in block: 850415
(T+0:08:14) ETH Rail Adapter publishes TR to Ethereum URIB Settlement Contract: method:
anchor_commitment() call eth_anchor_tx:
"0xf9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d3c2b1a0..." mined in block: 22847380
(T+0:09:55) Commitment status → ANCHORED (both rails) taproot_root:
"9a2c7e5f3b1d8a4c6e2f9b3d1a7c5e4f..." written to commitment record. Log Entry #0005: {
from:ANCHORING, to:PENDING_FINALITY, event:"taproot_published", ts:"01:58:10.001Z" }

T+0:09.560

STATE: ANCHORING → PENDING_FINALITY

Finality Watcher
ETH activated: monitoring: block 22847380, tx "0xf9e8d7..." poll_method: beacon chain finality
subscription required: epoch finalization (~12.8 min) Finality Watcher BTC activated: monitoring:
block 850415, tx "a1b2c3..." poll_method: block confirmation depth query every 60s required: 6
confirmations (~60 min)

T+0:15.020

Ethereum Finality Confirmed

FW-ETH: Epoch
194 finalized. Block 22847380 in finalized chain. confirmation_depth: 2 epochs anchor_block_hash:
"0x1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2" FinalityAttestation (ETH)
written: { "attestation_id": "fa:urib:v1:3e7a9c2f1b5d8a4c...", "rail_id": "ethereum:mainnet",
"anchor_tx_hash": "0xf9e8d7c6b5a4f3e2...", "anchor_block_height": 22847380,
"confirmation_depth": 2, "finality_model": "epoch-finalization", "attested at": "2026-07-
09T02:13:01.000Z", "watcher_sig": "base64url:MEQCIA7f3a1c9e..." } Commitment status (ETH
side) → PROVEN

T+1:05.340

Bitcoin Finality Confirmed

FW-BTC: Block

850421 mined (6 blocks above anchor 850415). confirmation_depth: 6 block 850421 hash: "0000000000000000000029c1e4a7f3b2d5e8a1c4f7b3d9e2a5c8f1b4d7e3a9c2f" FinalityAttestation (BTC) written: { "attestation_id": "fa:urib:v1:7b2d5e8a1c4f7b3d...", "rail_id": "bitcoin:mainnet", "anchor_tx_hash": "a1b2c3d4e5f6a7b8c9d0e1f2...", "anchor_block_height": 850415, "anchor_block_hash": "000000000000000000002abc1e...", "confirmation_depth": 6, "finality_model": "6-block-confirmation", "rpc_endpoints_used": ["https://btc-rpc-1.urib.internal", "https://btc-rpc-2.urib.internal"], "attested_at": "2026-07-09T03:03:21.000Z", "watcher_sig": "base64url:MEUCIQDb9a3f1c7e..." } All commitments → FINALIZED Log Entry #0006: { from:PENDING_FINALITY, to:FINALIZING, event:"all_finalities_confirmed", ts:"03:03:21.123Z" }

T+1:05.350

STATE: PENDING_FINALITY → FINALIZING

Proof assembly begins. All components collected: - commitment_leaves: ["e5d3f1a7c2b9e4f60c3e7b1d9f2a4c6..."] - taproot_root: "9a2c7e5f3b1d8a4c6e2f9b3d1a7c5e4f..." - ordinal_proof: (signed by Ordinal Oracle) - rail_anchors: { bitcoin:mainnet: {...}, ethereum:mainnet: {...} } - identity_cont_proof: null (S2, not S5) Collecting witness signatures: - Settlement Escrow sig: "base64url:MEQCIAx3f1a7c9e..." ✓ - Taproot Aggregator sig: "base64url:MEUCIQDe9c3f7a1b..." ✓ - Ordinal Oracle sig: "base64url:MEQCIBf7a3c9e1b..." ✓ proof_hash = SHA3_256(serialized_proof_without_hash_field) = "7f3a1c9e2b5d8f4a3c6e1f9b2d5a8c4e7f3b1d9a2c5e8f4b7d3a1c9e2b5d8f4a"

T+1:05.383

STATE: FINALIZING → SETTLED

Final Settlement Proof: { "proof_id": "sp:urib:v1:7f3a1c9e2b5d8f4a...", "settlement_id": "b7e3c1a9-4f2d-4e8a-b0c6-d3f7a2e9c1b4", "canonical_id": "c-hash:sha3-256:e7a3c1f9b2d4e6a8...", "settlement_category": "S2", "taproot_root": "9a2c7e5f3b1d8a4c6e2f9b3d1a7c5e4f...", "commitment_leaves": ["e5d3f1a7c2b9e4f60c3e7b1d9f2a4c6..."], "merkle_path": [], // single-leaf tree; empty path "ordinal_proof": { "ordinal": {"rail_id": "bitcoin:mainnet", "epoch": 842, "seq": 50017}, "prev_sequence": 50016, "oracle_sig": "base64url:...", "rail_anchors": { "bitcoin:mainnet": { "block_height": 850415, "tx_hash": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0...", "finality_status": "FINALIZED", "confirmation_depth": 6 }, "ethereum:mainnet": { "block_height": 22847380, "tx_hash": "0xf9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0...", "finality_status": "FINALIZED", "confirmation_depth": 2 } }, "identity_continuity_proof": null, "witness_sigs": ["base64url:MEQCIAx3f1a7c9e2b5d8f4a3c6e1f9b2d5a8c...", "base64url:MEUCIQDe9c3f7a1b2d4e6f8a0c3b7e1d9f2a4...", "base64url:MEQCIBf7a3c9e1b2d4f6a8c0e3b7d1f9a2c4e..."], "settled_at": "2026-07-09T03:03:21.383Z", "proof_hash": "7f3a1c9e2b5d8f4a3c6e1f9b2d5a8c4e7f3b1d9a2c5e8f4b7d3a1c9e2b5d8f4a" } Log Entry #0007: { from:FINALIZING, to:SETTLED, event:"proof_assembled", ts:"2026-07-09T03:03:21.383Z" } Total elapsed time: 65 minutes, 21.383 seconds Bottleneck: Bitcoin 6-block confirmation (60 min 20 sec)

Appendix C — Taproot Leaf Hash Reference Implementation (Pseudocode)

```
// Taproot Leaf Hash Computation — Reference Implementation // Computes the leaf hash for a single
BridgeCommitment, as used // by the Taproot Aggregator in Section 9.2. function
compute_taproot_leaf_hash(commitment: BridgeCommitment) -> bytes32: // Serialize commitment
fields in deterministic canonical order leaf_content = canonical_serialize({ "commitment_id":
commitment.commitment_id, "state_hash_from": commitment.state_hash_from, "state_hash_to":
commitment.state_hash_to, "ordinal_from": commitment.ordinal_from, // { rail_id, epoch, sequence
} "ordinal_to": commitment.ordinal_to, "rail_from": commitment.rail_from, "rail_to":
commitment.rail_to }) // Apply tagged hash (domain-separated to prevent cross-protocol attacks) tag
= SHA3_256("URIB/TaprootLeaf/v1") leaf_hash = SHA3_256(tag || tag || leaf_content) return
leaf_hash // Taproot Root Computation (N leaves) function compute_taproot_root(commitments:
list[BridgeCommitment]) -> bytes32: // Step 1: Compute leaf hashes leaf_hashes =
[compute_taproot_leaf_hash(c) for c in commitments] // Step 2: Sort lexicographically (ensures
determinism regardless of input order) sorted_leaves = lexicographic_sort(leaf_hashes) // Step 3: Build
Merkle tree bottom-up return merkle_root(sorted_leaves) function merkle_root(leaves: list[bytes32]) ->
bytes32: if len(leaves) == 0: raise ValueError("Cannot compute Merkle root of empty leaf set") if
len(leaves) == 1: // Single leaf: root = SHA3_256(leaf || leaf) for canonical consistency return
SHA3_256(leaves[0] || leaves[0]) // Pad to even count by duplicating last element if len(leaves) % 2
== 1: leaves = leaves + [leaves[-1]] // Recursive pairwise hashing parent_layer = [] for i in range(0,
len(leaves), 2): left = leaves[i] right = leaves[i+1] // Sort pair to ensure H(a,b) == H(b,a) —
commutative Merkle tree if left > right: left, right = right, left parent = SHA3_256(left || right)
parent_layer.append(parent) return merkle_root(parent_layer) // Merkle Path Generation (for proof
inclusion) function generate_merkle_path(target_leaf_hash: bytes32, all_leaf_hashes: list[bytes32])
-> list[MerklePathNode]: leaves = lexicographic_sort(all_leaf_hashes) if len(leaves) % 2 == 1:
leaves = leaves + [leaves[-1]] path = [] target = target_leaf_hash while len(leaves) > 1: if
len(leaves) % 2 == 1: leaves = leaves + [leaves[-1]] next_layer = [] for i in range(0, len(leaves),
2): left = leaves[i] right = leaves[i+1] if target == left:
path.append(MerklePathNode(sibling=right, position="right")) target = SHA3_256(min(left,right) ||
max(left,right)) elif target == right: path.append(MerklePathNode(sibling=left, position="left"))
target = SHA3_256(min(left,right) || max(left,right)) sorted_pair = (min(left,right), max(left,right))
next_layer.append(SHA3_256(sorted_pair[0] || sorted_pair[1])) leaves = next_layer return path
```

Appendix D — Ordinal Proof Reference Implementation (Pseudocode)

```
// Ordinal Proof Reference Implementation // Used by the Ordinal Oracle to issue and verify ordinal
proofs. struct Ordinal: rail_id: string epoch: uint64 sequence: uint64 struct OrdinalProof:
ordinal: Ordinal prev_sequence: uint64 issued_at: timestamp oracle_sig: bytes // Ed25519
signature // Ordinal Issuance (by Ordinal Oracle) function issue_ordinal_proof( ordinal: Ordinal,
canonical_id: string, oracle_key: Ed25519PrivateKey) -> OrdinalProof: // Verify monotonicity
last_seq = registry.get_last_sequence(ordinal.rail_id, canonical_id) if ordinal.sequence <= data-
id="2536" last_seq: raise OrdinalViolation(f"sequence {ordinal.sequence} not > last {last_seq}") //
Verify epoch freshness current_epoch = get_current_epoch(ordinal.rail_id) if ordinal.epoch <
current_epoch - MAX_EPOCH_LAG: raise OrdinalViolation(f"epoch {ordinal.epoch} is stale") //
Record in registry (atomic write) registry.record(ordinal.rail_id, canonical_id, ordinal.sequence) proof
= OrdinalProof( ordinal = ordinal, prev_sequence = last_seq, issued_at = now() ) // Sign
the proof proof_bytes = canonical_serialize(proof) proof.oracle_sig = Ed25519_sign(oracle_key,
SHA3_256(proof_bytes)) return proof // Ordinal Proof Verification (by Settlement Engine at validation
time) function verify_ordinal_proof( proof: OrdinalProof, oracle_pubkey: Ed25519PublicKey) ->
bool: // Step 1: Verify oracle signature proof_content = canonical_serialize({ "ordinal":
proof.ordinal, "prev_sequence": proof.prev_sequence, "issued_at": proof.issued_at }) if NOT
Ed25519_verify(oracle_pubkey, SHA3_256(proof_content), proof.oracle_sig): return false //
ORDINAL_PROOF_INVALID: oracle signature bad // Step 2: Verify sequence monotonicity if
proof.ordinal.sequence <= data-id="2537" proof.prev_sequence: return false //
ORDINAL_PROOF_INVALID: not monotonically increasing // Step 3: Verify epoch freshness if
proof.ordinal.epoch < get_current_epoch(proof.ordinal.rail_id) - MAX_EPOCH_LAG: return false //
ORDINAL_PROOF_INVALID: epoch too old // Step 4: Verify not in nullifier registry if
registry.is_nullified(proof.ordinal, canonical_id): return false // ORDINAL_PROOF_INVALID:
ordinal has been nullified return true // Nullifier Issuance (on REVERTED settlement) function
issue_nullifier(ordinal: Ordinal, canonical_id: string) -> void: nullifier = { "ordinal": ordinal,
"canonical_id": canonical_id, "nullified_at": now(), "permanent": true // nullifiers never expire }
nullifier_registry.write(nullifier) // atomic; throws on conflict // After this call, any future settlement
using this (ordinal, canonical_id) // pair will fail at COMMIT-3 / ORD-2 checks.
```

Appendix E — Settlement Glossary Extension

The following terms are introduced in this Settlement Flow Specification and are not defined in the URIB Canonical Architecture Master Specification v1.0. All terms defined in the Master Spec are inherited and apply without redefinition here.

Term	Definition
Settlement Engine	The URIB subsystem responsible for orchestrating the complete settlement lifecycle. It is stateful, durable, idempotent, and observable. It implements the state machine defined in Section 4 and exposes the API defined in Section 13.
Settlement Escrow	A smart contract or URIB protocol primitive that holds canonical object state during in-flight settlement. It issues bridge commitments, manages the exclusive lock on the canonical object, and executes reversion when required.
Finality Watcher	A per-rail, per-settlement component that continuously monitors a rail for finality confirmation of a published Taproot Root anchor transaction. Upon confirming finality, it writes a signed Finality Attestation to the bridge commitment record.
Dispute Arbitrator	The component or DAO that receives disputed settlements, evaluates the original commitment against the challenger's counter-proof, and issues a binding, signed, deterministic resolution (VALID or INVALID).
Challenge Bond	A financial deposit (in a specified token on a specified rail) that a challenger must lock in the Settlement Escrow before submitting a dispute challenge. If the challenger is correct, the bond is returned. If the challenger is incorrect, the bond is slashed and distributed to the Initiator and the URIB protocol treasury.
Nullifier	A permanent, irrevocable record in the ordinal registry that marks a specific (ordinal, canonical_id) pair as consumed and ineligible for reuse. Nullifiers are issued upon settlement reversion to prevent replay attacks and double-settlement attempts using the reverted ordinal.
Optimistic Window	The time period following entry into the OPTIMISTIC state during which any party may submit a dispute challenge against the settlement. Defined by <code>challenge_window_seconds</code> in the Settlement Request options. After expiry with no valid challenge, the settlement finalizes automatically.
Re-anchor	The act of re-submitting a Taproot Root publication transaction to a rail after the original anchor transaction was orphaned, replaced (RBF), or evicted. Re-anchoring uses the same TR (deterministic) but creates a new on-chain transaction. Tracked by the <code>re_anchor_attempt[rail_id]</code> counter.
Resumption	The process by which the Settlement Engine recovers an in-flight settlement after an unexpected interruption (engine crash, restart, or maintenance). Resumption is safe and idempotent because all settlement state is durably persisted and all operations are idempotent (keyed by <code>settlement_id</code>).
Settlement Funnel	A monitoring view showing the count of settlements in each state at a given point in time, visualized as a funnel or pipeline to identify drop-off rates at each stage. Used for operational observability (see Section 12.4).

Term	Definition
Proof Hash	A SHA-3-256 hash of the complete Settlement Proof document (with the <code>proof_hash</code> field itself set to the empty string before hashing). The <code>proof_hash</code> provides tamper-evident self-authentication of the Settlement Proof: any modification to any field invalidates the <code>proof_hash</code> , which is detectable at verification Step 1 (Section 10.1).
Commitment Leaf	A single SHA-3-256 tagged hash derived from the contents of one BridgeCommitment record. Commitment leaves are the input data to the Taproot Aggregator's Merkle tree computation. The full set of commitment leaves for a <code>settlement_id</code> , together with their Merkle tree, constitutes the Taproot Root.
Finality Attestation	A signed record produced by a Finality Watcher confirming that a specific Taproot Root anchor transaction has reached the URIB-required confirmation depth on a specific rail. The Finality Attestation is a required component of the Settlement Proof and is verified at proof verification Step 2 (Section 10.1).
Rail-Specific Finality	The finality model and confirmation requirement applied to a specific rail by URIB. Defined per rail in Section 6. Examples: 6 block confirmations (Bitcoin), epoch finalization (Ethereum), 2 validated ledger closes (XRPL), 32-slot confirmation (Solana).

Appendix F — Version History & Changelog Template

Version	Date	Author	Status	Summary of Changes
1.0.0	2026-07-09	URIB Protocol Working Group	NORMATIVE / RELEASED	Initial release. Defines complete settlement lifecycle for URIB v1.0. Covers S1–S6 categories, all states, all transitions, all data structures, 6 rail paths, full API, security model, and conformance requirements.
1.0.1	TBD	—	DRAFT	[Reserved for minor corrections and clarifications. No normative changes.]
1.1.0	TBD	—	PLANNED	[Planned: ZK-SNARK settlement proof normative specification (Section 10.4 promoted from informational). Additional rail paths: Cosmos IBC, Polkadot XCMP, StarkNet.]
2.0.0	TBD	—	PLANNED	[Planned: Multi-hop settlement (S7); recursive settlement (nested canonical objects); post-quantum signature migration (Ed448 or Dilithium).]

Changelog Template for Future Versions

v[VERSION] — [DATE] ### Normative Changes - [BREAKING] Section N.N: [Description of breaking change] - [ADDED] Section N.N: [New normative requirement] - [REMOVED] Section N.N: [Removed requirement and rationale] - [CHANGED] Section N.N: [Modified requirement — old behavior → new behavior] ### Non-Normative Changes - [CLARIFIED] Section N.N: [Clarification without behavior change] - [EDITORIAL] [Spelling, formatting, example corrections] ### Test Vector Updates - [UPDATED] Appendix B: [New worked example data] - [ADDED] Appendix B: [New scenario] ### Migration Notes (for breaking changes) - Implementations MUST [specific action] before upgrading to this version. - Backward compatibility: [yes/no/conditional]

Appendix G — Cross-Reference Index to URIB Canonical Architecture Master Specification

The following table maps each major section of this Settlement Flow Specification (SFS) to the corresponding section(s) of the URIB Canonical Architecture Master Specification v1.0 (MAS) where the underlying concept is defined. Implementors of the Settlement Engine MUST be familiar with all referenced MAS sections before implementing the corresponding SFS sections.

Settlement Flow Spec (SFS) Section	Master Architecture Spec (MAS) Section	Relationship
§1.1 — What Settlement Means in URIB	MAS §Bridge Commitment Lifecycle MAS §Universal Bridge Stack Layer 5	SFS extends the Bridge Commitment Lifecycle defined in MAS with the full terminal settlement phase.
§1.2 — Settlement Categories (S1–S6)	MAS §Rail Mapping (φ) MAS §Cross-Rail Invariants	Rail mapping φ determines which rail-pairs are valid for S2/S3. Cross-rail invariants are the pre-conditions SFS checks at VALIDATING.
§2 — Settlement Actors & Roles	MAS §Trust Model MAS §Component Architecture	SFS actors inherit trust levels from MAS trust model. Component boundaries align with MAS architecture.
§3.1 — Settlement Request (canonical_id, semantic_id, ordinal)	MAS §C-Hash Derivation MAS §Semantic ID (SID) Registry MAS §Ordinal System	canonical_id uses the C-Hash algorithm from MAS. semantic_id follows SID format from MAS. ordinal structure defined in MAS Ordinal System.
§3.2 — Bridge Commitment (Settlement Context)	MAS §Bridge Commitment Schema MAS §Layer 4: Commitment Layer	SFS BridgeCommitment extends the MAS canonical Bridge Commitment with settlement_id FK and settlement-specific status fields.
§3.3 — Settlement Proof (taproot_root, commitment_leaves)	MAS §Taproot Root (TR) Definition MAS §Layer 3: Semantic Anchoring	Taproot Root construction and semantic anchoring semantics are defined in MAS. SFS defines the Settlement Proof artifact that wraps the TR and all attestations.

Settlement Flow Spec (SFS) Section	Master Architecture Spec (MAS) Section	Relationship
§4 — Settlement State Machine	MAS §Bridge Commitment Status Enum MAS §Protocol State Lifecycle	MAS defines PENDING, ANCHORED, FINALIZED as commitment-level statuses. SFS defines the full settlement-level state machine that orchestrates those transitions.
§5.5 — S5 Identity Continuity Settlement	MAS §ThreadZero MAS §Identity Lifecycle Events MAS §Layer 1: Identity Layer	ThreadZero genesis, key rotation, fork, and tombstone events are defined in MAS. SFS defines how those events are settled across rails.
§6 — Rail-Specific Settlement Paths	MAS §Rail Adapters MAS §Supported Rails Registry MAS §Rail Capability Declarations	Rail registry, Rail Adapter interface, and capability declarations are defined in MAS. SFS adds settlement-specific finality requirements and error conditions for each rail.
§8 — Settlement Validation (CANON-1, ORD-1, etc.)	MAS §C-Hash Algorithm MAS §Ordinal Monotonicity Invariant MAS §Non-Duplication Invariant MAS §Cross-Rail Invariants	The invariants checked at SFS VALIDATING phase are formally defined in MAS. SFS operationalizes those invariants as a numbered checklist with error codes.
§9.2 — Taproot Root Computation	MAS §Taproot Root Definition MAS §Merkle Construction MAS §Layer 3: Anchoring Layer	The Taproot Root computation algorithm, Merkle tree construction, and anchoring mechanism are defined in MAS. SFS applies them specifically in the settlement anchoring context.
§10 — Settlement Proof Verification	MAS §Proof Verifiability Requirements MAS §Light Client Protocol	MAS defines the verifiability requirements for URIB proofs. SFS provides the concrete verification algorithm and all failure codes for Settlement Proofs specifically.
§14 — Security Considerations	MAS §Security Model MAS §Adversarial Assumptions MAS §Cryptographic Primitives (SHA-3-256, Ed25519)	MAS defines the adversarial threat model and cryptographic primitive choices. SFS extends the security analysis specifically for the settlement flow attack surfaces.
Appendix C — Taproot Leaf Hash	MAS §Tagged Hash Construction MAS §Domain Separation	Tagged hash and domain separation pattern defined in MAS. SFS provides the leaf-level instantiation for bridge commitment hashing.
Appendix D — Ordinal Proof	MAS §Ordinal Oracle MAS §Ordinal Registry MAS §Monotonicity Proof	Ordinal Oracle, registry structure, and monotonicity proof format defined in MAS. SFS provides the reference implementation for ordinal proof issuance and verification in the settlement context.

Note to Implementors:

This Settlement Flow Specification is designed to be self-contained for implementors who have already read the URIB Canonical Architecture Master Specification v1.0. Any concept referenced herein without full definition SHALL be resolved by consulting the corresponding MAS section in the cross-reference table above. In case of any conflict between this SFS and the MAS on a topic within the settlement domain, this SFS takes precedence. On all other topics, the MAS is authoritative.

Document: URIB Settlement Flow Specification v1.0.0

Companion: URIB Canonical Architecture Master Specification v1.0

Status: NORMATIVE

Classification: Developer & Integrator Reference

Issued: 2026-07-09

License:

This document uses normative language per RFC 2119. The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL are to be interpreted as described in RFC 2119.